

华中科技大学

2025

硬件综合训练

课程设计报告

题 目： 5 段流水 CPU 设计
专 业： 计算机科学与技术
班 级： 计算机本硕博 2301
学 号： U202315764
姓 名： 王希宇
电 话： 18843349011
邮 件： 3128960679@qq.com

华中科技大学课程设计报告

目 录

1	课程设计概述.....	3
1.1	课设目的	3
1.2	设计任务	3
1.3	设计要求	3
1.4	技术指标	4
2	总体方案设计.....	6
2.1	单周期 CPU 设计	6
2.2	中断机制设计.....	10
2.3	流水 CPU 设计.....	13
2.4	气泡式流水线设计.....	14
2.5	重定向流水线设计.....	15
2.6	动态分支预测机制.....	15
3	详细设计与实现.....	17
3.1	单周期 CPU 实现	17
3.2	中断机制实现.....	24
3.3	流水 CPU 实现.....	28
3.4	气泡式流水线实现.....	30
3.5	重定向流水线实现.....	31
3.6	动态分支预测机制实现	33
4	实验过程与调试.....	38
4.1	测试用例和功能测试.....	38
4.2	性能分析	40
4.3	主要故障与调试.....	41
4.4	实验进度	49

华中科技大学课程设计报告

5	团队任务	50
5.1	整体玩法概述.....	50
5.2	核心组成逻辑.....	51
5.3	组员分工情况.....	59
6	设计总结与心得	60
6.1	课设总结	60
6.2	课设心得	60
	参考文献.....	62

1 课程设计概述

1.1 课设目的

计算机组成原理是计算机专业的核心基础课。该课程力图以“培养学生现代计算机系统设计能力”为目标，贯彻“强调软/硬件关联与协同、以 CPU 设计为核心/层次化系统设计的组织思路，有效地增强对学生的计算机系统设计及实现能力的培养”。课程设计是完成该课程并进行了多个单元实验后，综合利用所学的理论知识，并结合在单元实验中所积累的计算机部件设计和调试方法，设计出一台具有一定规模的指令系统的简单计算机系统。所设计的系统能在 LOGISIM 仿真平台和 FPGA 实验平台上正确运行，通过检查程序结果的正确性来判断所设计计算机系统正确性。

课程设计属于设计型实验，不仅锻炼学生简单计算机系统的设计能力，而且通过进行中央处理器底层电路的实现、故障分析与定位、系统调试等环节的综合锻炼，进一步提高学生分析和解决问题的能力。

1.2 设计任务

本课程设计的总体目标是利用 FPGA 以及相关外围器件，设计五段流水 CPU，要求所设计的流水 CPU 系统能支持自动和单步运行方式，能正确地执行存放在主存中的程序的功能，对主要的数据流和控制流通过 LED、数码管等适时的进行显示，方便监控和调试。尽可能利用 EDA 软件或仿真软件对模型机系统中各部件进行仿真分析和功能验证。在学有余力的前提下，可进一步扩展相关功能。

1.3 设计要求

- (1) 根据课程设计指导书的要求，制定出设计方案；
- (2) 分析指令系统格式，指令系统功能。
- (3) 根据指令系统构建基本功能部件，主要数据通路。
- (4) 根据功能部件及数据通路连接，分析所需要的控制信号以及这些控制信号的有效形式；
- (5) 设计出实现指令功能的硬布线控制器；

华中科技大学课程设计报告

- (6) 调试、数据分析、验收检查;
- (7) 课程设计报告和总结。

1.4 技术指标

- (1) 支持表 1.1 前 27 条基本 32 位 MIPS 指令;
- (2) 支持教师指定的 4 条扩展指令;
- (3) 支持多级嵌套中断, 利用中断触发扩展指令集测试程序;
- (4) 支持 5 段流水机制, 可处理数据冒险, 结构冒险, 分支冒险;
- (5) 能运行由自己所设计的指令系统构成的一段测试程序, 测试程序应能涵盖所有指令, 程序执行功能正确。
- (6) 能运行教师提供的标准测试程序, 并自动统计执行周期数
- (7) 能自动统计各类分支指令数目, 如不同种类指令的条数、冒险冲突次数、插入气泡数目、load-use 冲突次数、动态分支预测流水线能自动统计预测成功与失败次数。

表 1.1 指令集

#	指令助记符	简单功能描述	备注
1	ADD	加法	
2	ADDI	立即数加	
3	ADDIU	无符号立即数加	
4	ADDU	无符号数加	
5	AND	与	
6	ANDI	立即数与	指令格式参考 MIPS32 指令集, 最终功能以 MARS
7	SLL	逻辑左移	模拟器为准。
8	SRA	算数右移	
9	SRL	逻辑右移	
10	SUB	减	
11	OR	或	
12	ORI	立即数或	
13	NOR	或非	

华中科技大学课程设计报告

#	指令助记符	简单功能描述	备注
14	LW	加载字	
15	SW	存字	
16	BEQ	相等跳转	
17	BNE	不相等跳转	
18	SLT	小于置数	
19	STI	小于立即数置数	
20	SLTU	小于无符号数置数	
21	J	无条件转移	
22	JAL	转移并链接	
23	JR	转移到指定寄存器	If \$v0==10 halt(停机指令)
24	SYSCALL	系统调用	else 数码管显示\$a0 值
25	MFC0	访问 CP0	中断相关，可简化，选做
26	MTC0	访问 CP0	中断相关，可简化，选做
27	ERET	中断返回	异常返回，选做
28	SRA	算数右移	
29	LUI	高位立即数加载	16 位立即数加载到寄存器
30	LB	字节加载	的高 16 位，低 16 位清零
31	BEGU	无符号数大于等于跳转	

2 总体方案设计

2.1 单周期 CPU 设计

在本次硬件综合训练课程设计中，单周期 CPU 设计旨在构建一个能够在单个时钟周期内完成指令取指、译码、执行、访存及写回全过程的 RISC-V 处理器。设计围绕 32 位 RISC-V 指令集展开，重点支持包括算术逻辑运算、访存操作、分支跳转及系统调用在内的 24 条基本指令，并在此基础上扩展的 4 条 CCAB 系列指令。CPU 的设计采用硬布线的控制方式、指令存储器和数据存储器分离的哈佛结构。设计流程为构建统一的数据通路，设计硬布线控制器，在 Logisim 仿真平台中进行系统调试，最后进行差异化指令的支持。

总体结构图如图 2.1 所示。

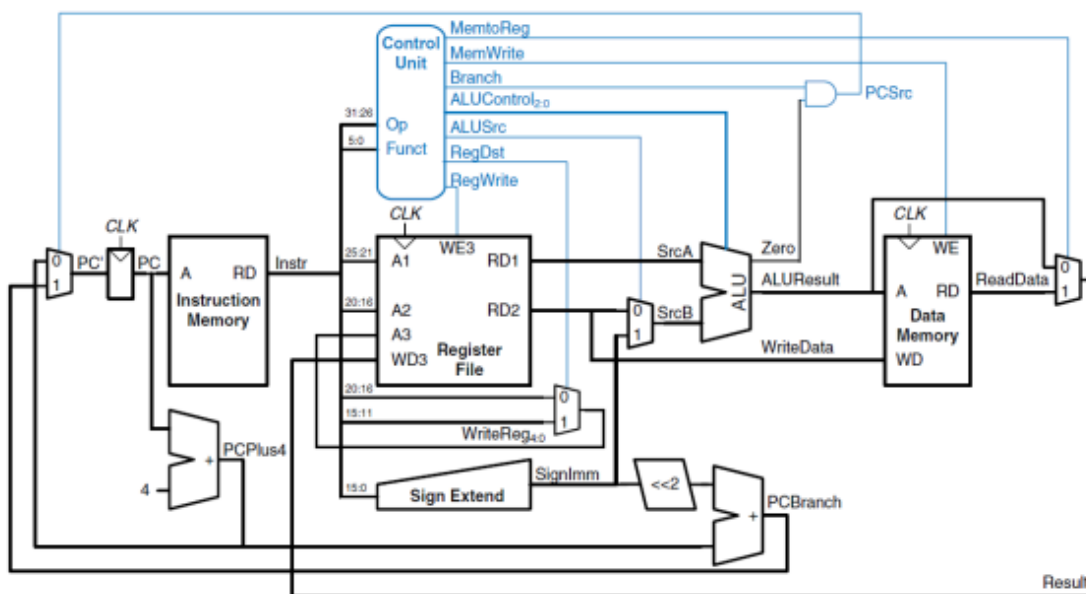


图 2.1 总体结构图

2.1.1 主要功能部件

1. 程序计数器 PC

程序计数器使用一个 32 位的寄存器实现，其核心功能是指令地址的存储与更新。在每个时钟周期的上升沿，PC 会将当前输出的地址锁存，并将其送入指令存储器以

华中科技大学课程设计报告

读取当前指令。同时，通过一个专用的加法器，将 PC 当前值加 4（因为 RISC-V 指令字长为 4 字节），计算出顺序执行的下一条指令地址，并作为 PC 在下一个时钟周期的默认输入，从而实现程序的顺序执行。对于分支和跳转指令，该输入会被多路选择器切换为相应的目标地址。

2. 指令存储器 IM

指令存储器使用一个只读存储器（ROM）组件实现，其数据宽度为 32 位，用于存储机器指令。它接收来自 PC 的地址信号，并在一个时钟周期内异步地输出该地址对应的 32 位指令字。在 Logisim 仿真中，我们通过加载由 RARS 汇编器生成的 .hex 文件来初始化 ROM 内容，从而将测试程序固化在 CPU 中。

3. 运算器

本设计直接复用已实现的 32 位算术逻辑单元（ALU）。该 ALU 接收来自寄存器堆和立即数生成器的两个 32 位操作数，并根据控制器产生的 ALUOp 控制信号，选择执行加法、减法、逻辑运算、移位等多种算术逻辑操作。其运算结果将用于后续的寄存器写回、数据存储器地址计算或分支条件判断。

4. 寄存器堆 RF

寄存器堆是包含 32 个 32 位寄存器的存储单元。它提供两个读端口和一个写端口，支持同时读取两个源寄存器（由指令中的 rs1 和 rs2 字段选择）和一个目的寄存器（由 rd 字段选择）的写入操作。写操作由写使能信号 RegWrite 控制，仅在时钟上升沿且使能有效时，才会将数据（来自 ALU 运算结果或数据存储器）写入目标寄存器。

5. 数据存储器 DM

数据存储器使用随机存取存储器（RAM）组件实现，用于存储程序运行时的数据。它接收 ALU 计算出的有效地址，并根据控制信号进行写操作：当 MemWrite 信号有效时，在时钟上升沿将指定数据写入对应地址，并且总是从给定地址读取数据。为了支持差异化指令中的字节、半字访问指令，该 RAM 组件配备了字节使能选择逻辑。

2.1.2 数据通路的设计

单周期 CPU 的数据通路设计围绕指令的执行流程展开，其核心思想是为不同功能的指令构建统一的硬件执行路径，并通过多路选择器和控制器信号来协调指令间的差异。整体设计采用迭代方式，从简单指令开始，逐步扩展通路以支持更复杂的指令。

1. 指令取指通路

这是所有指令的公共起点。程序计数器（PC）指向当前指令地址，从指令存储器（IM）中取出 32 位指令。同时，一个**独立的加法器**将 PC 值+4，计算下一条顺序指令的地址，并默认写回 PC，为下个时钟周期做准备。

2. 指令译码与寄存器读通路

取出的指令字中固定的字段（如 rs1,rs2,rd）直接连接到寄存器堆（RF）的对应端口，从而同时读出两个源操作数 R[rs1]和 R[rs2]的值。指令中的立即数字段被送入**立即数生成器（ImmGen）**，根据指令类型（I 型、S 型、B 型等）进行符号扩展并重组，生成统一的 32 位立即数。

3. 指令执行通路

这是数据通路的核心分流点。

算术逻辑运算指令（R 型、I 型）：通过一个多路选择器（由 ALUSrc 信号控制），ALU 的第二个操作数可以选择来自寄存器堆的 R[rs2]或来自立即数生成器的扩展立即数。ALU 根据 ALUOp 信号对两个操作数进行指定运算，结果将用于写回寄存器。

访存指令（Load/Store）：ALU 同样用于计算有效地址，其运算数为 R[rs1]与扩展后的立即数之和。对于存储指令（SW），需要将 R[rs2]的值写入数据存储器（DM）；对于加载指令（LW），则需要从 DM 读出数据。

4. 结果写回通路

运算或加载的结果需要写回寄存器堆，这里存在多个来源，因此引入了另一个关键的多路选择器（由 MemtoReg 信号控制）。

华中科技大学课程设计报告

当 MemtoReg=0 时，选择 ALU 的运算结果写回寄存器（用于 R 型、I 型运算指令）。

当 MemtoReg=1 时，选择从数据存储器（DM）读出的数据写回寄存器（用于 Load 指令）。

5. 程序控制通路

为了支持分支和跳转指令（如 BEQ, JAL），在 PC 的输入前端增加了几个多路选择器。该选择器不再仅仅选择 PC+4，还可以选择分支目标地址或跳转地址，从而实现程序的非顺序执行。分支判断（如比较 R[rs1]和 R[rs2]是否相等）通常在 ALU 中完成，并由控制器产生相应的分支控制信号。

此外还有另一种数据通路的设计思想，是基于指令类型进行独立构建。该方法首先为每一条指令从头至尾地规划其完整的数据通路，随后通过交叉比对，在不同指令的数据通路之间复用重叠的功能部件，从而整合出一个统一且高效的全局数据通路。

2.1.3 控制器的设计

控制器的设计是单周期 CPU 的核心，它负责将指令代码翻译成控制数据通路运作的具体信号。如果说数据通路是一个城市的车流，那控制器就是城市的交通信号灯。本设计采用**硬布线控制器**，其本质是一个大型的组合逻辑电路，根据当前指令的操作码（Opcode）和功能码（Funct3, Funct7）字段，即时产生所有必需的控制信号，以驱动数据通路正确执行指令。

控制器输入输出见表 2.1 控制器输入输出。

表 2.1 控制器输入输出

引脚	类型	位宽	功能说明
OP	输入	5	指令字 Opcode 字段中的 5 位, IR[6:2]
Funct	输入	8	指令字 Funct3/7 字段,IR[30,25,14:12]
IR[21]	输入	1	指令字 21 位,用于区分 ecall 与 uret
JAL	输出	1	JAL 指令译码信号
JALR	输出	1	JAL 指令译码信号

华中科技大学课程设计报告

引脚	类型	位宽	功能说明
Beq	输出	1	<i>Beq</i> 指令译码信号
Bne	输出	1	<i>Bne</i> 指令译码信号
ecall	输出	1	<i>ecall</i> 指令译码信号
S_Type	输出	1	<i>S</i> 型指令译码信号
MemToReg	输出	1	写回数据选择, 为1 写回存储器数据
MemWrite	输出	1	存储器写使能, 高电平有效
AluOp	输出	4	<i>ALU</i> 运算选择
AluSrcB	输出	1	<i>ALU</i> 的第二输入来源选择控制
RegWrite	输出	1	寄存器堆写使能, 高电平有效
URET	输出	1	<i>URET</i> 指令译码信号 (<i>ccab</i>)
U_type	输出	1	<i>U</i> 型指令译码信号 (<i>ccab</i>)
Byte	输出	1	只读一个字节 (<i>ccab</i>)
BEGU	输出	1	<i>BEGU</i> 指令译码信号 (<i>ccab</i>)

2.2 中断机制设计

2.2.1 总体设计

中断机制是 CPU 响应外部异步事件的关键。本设计为已实现的 CPU 增加了中断处理能力, 支持 3 个由按键触发的外部中断源, 其优先级为 $1 < 2 < 3$ 。设计遵循 RISC-V 规范并进行了适当简化, 实现了单级中断和多级嵌套中断的功能。本次设计采用 EPC 硬件堆栈保护的中断, 将执行中断前的 PC 值放入 EPC 寄存器中进行保存。

2.2.2 硬件设计

关键硬件有下面几个:

1. **中断请求寄存器:** 锁存 3 个外部按键产生的中断请求信号, 并通过 LED 指示灯显示请求状态。
2. **中断使能寄存器:** 作为一个全局开关, 控制 CPU 是否响应任何中断请求。

- 3. 异常程序计数器：用于保存被中断指令的地址，即断点，确保中断返回后能继续正确执行。
- 4. 中断识别逻辑：采用优先编码器电路，根据固定的硬件优先级（1<2<3）仲裁同时发生的中断，并生成对应的中断编号。
- 5. 中断向量表：通过硬连线逻辑实现，将中断编号直接映射到固定的中断服务程序入口地址，简化了查找过程。

2.2.3 软件设计

采用硬 24 件堆栈保护 EPC 寄存器，只需要额外实现 3 条指令：URET，CSRRSI，CSRRC，功能如表 2.2 中断指令所示。

表 2.2 中断指令

指令	功能
URET	将 EPC 寄存器送 PC，开中断，发送中断结束信号熄灭当前中断请求的指示灯
CSRRSI	开中断
CSRRCI	关中断

2.2.4 中断流程

如图 2.2 中断流程图所示。

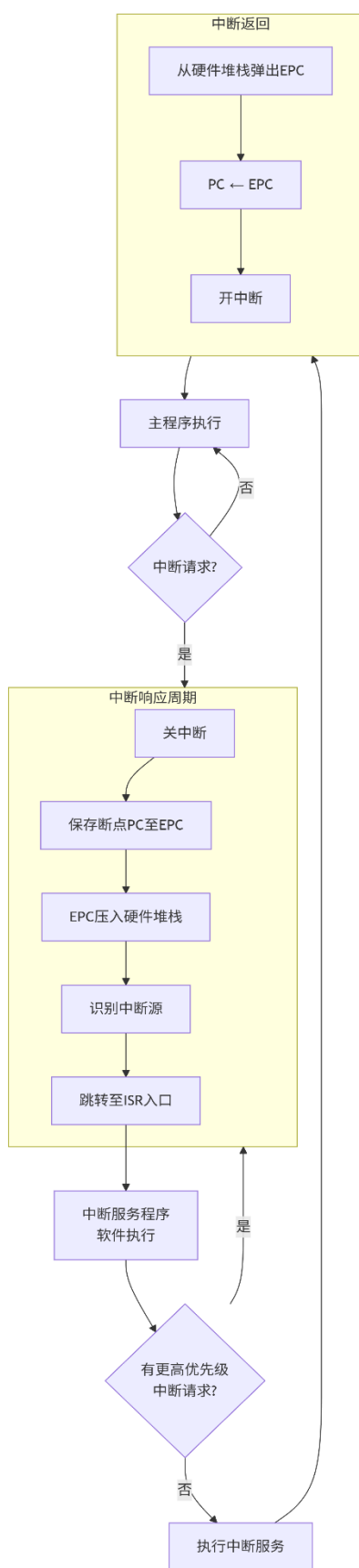


图 2.2 中断流程图

2.3 流水 CPU 设计

2.3.1 总体设计

本次流水 CPU 设计采用标准的 5 段流水线结构，将指令的执行过程划分为五个独立的阶段：**取指、译码、执行、访存和写回**。其核心设计思想是将单周期 CPU 中在一个时钟周期内完成的所有工作，拆分成五个耗时相近的阶段。每个阶段都在专用的部件中完成，并在时钟上升沿将结果连同必要的控制信号，通过**流水线接口寄存器**传递到下一阶段。这样，在同一个时钟周期内，最多可以有五条指令同时处于不同的执行阶段，从而极大地提高了处理器的吞吐率。

流水线 CPU 有三种冒险需要我们解决，如表 2.3 三种冒险所示，其中对数据冒险的不同解决办法诞生了气泡流水线和重定向流水线，分支冒险的第二种解决办法则诞生了动态分支预测流水线。

表 2.3 三种冒险

冒险	含义	解决办法
资源冒险	硬件资源不足，多指令同时争抢同一资源（主存，ALU）	增加部件消除（哈佛结构，加法器）
分支冒险	跳转指令改变 PC 值，使得流水线预取的后续指令无效	插入气泡，将预取的后续指令的流水接口清零即可；动态分支预测
数据冒险	指令间有数据依赖（前指令结果为后指令的操作数）	暂停后指令，前后指令间插入气泡；重定向技术

2.3.2 流水接口部件设计

流水接口部件是连接各流水段的桥梁，其核心是插入在每两个阶段之间的一组寄存器。这些寄存器负责在时钟上升沿锁存并传递前一阶段的所有输出信息，为下一阶段提供稳定的输入。此外还要有清零和暂停的功能以支持分支冒险和数据冒险的处理。

2.3.3 理想流水线设计

理想流水线是指在假设不存在任何数据冒险、结构冒险和控制冒险的理想情况下构建的 5 段流水线。在此设计中，我们默认：所有指令阶段数相同、每个阶段时延相同、无资源冲突、无段间互锁。

此阶段的主要工作是将单周期 CPU 的数据通路按阶段拆分，并插入上述流水线接口寄存器。同时，单周期 CPU 的控制器可以在 ID 段复用，但其产生的控制信号需要根据指令在流水线中的流动，被相应地锁存并传递到 EX、MEM 和 WB 段。

理想流水线是后续优化的基础，但其在实际运行测试程序时会因各种冒险而出现执行错误，因此需要进一步改进。

2.4 气泡式流水线设计

为解决理想流水线中实际存在的冒险问题，本设计首先引入了“气泡”机制。气泡是指在流水线中插入一个空操作指令。

1. **数据冒险处理**：主要解决 load-use 冲突。核心思想是暂停依赖冲突数据的指令，在数据冒险的两指令间插入气泡。如果是 ID 段 WB 段数据冒险，可以将寄存器堆设置为下跳沿写入，这样先写后读即可解决数据冒险；ID 段与 MEM 段数据冒险则要暂停 PC 和 IF/ID，给 ID/EX 插入气泡，如此下一时刻变为 ID 段 WB 段数据冒险；对于 ID 段 EX 段数据冒险，同样是暂停 PC 和 IF/ID，给 ID/EX 插入气泡，下一时刻变为 ID 段 MEM 段数据冒险。

2. **分支冒险处理**：对于分支指令（如 beq, bne），其目标地址和跳转条件在 EX 段才能确定。因此，当 ID 段识别出分支指令时，要根据分支判断结果决定是继续执行顺序指令还是冲刷掉已取入的错误指令（给 IF/ID 和 ID/EX 流水接口插入气泡）。

此设计通过增加冒险检测单元和气泡插入逻辑，保证了流水线功能的正确性，但

频繁的暂停会降低流水线的效率。

2.5 重定向流水线设计

为了减少因数据冒险而插入气泡带来的性能损失，本设计进一步引入了**重定向**机制。其核心思想是：**将后续指令所需的数据，在其产生后立即从更靠后的流水段“转发”到前面需要它的 ALU 输入端**，从而无需等待该数据写回寄存器堆。

1. 转发源与目标：

- **转发源**：来自 EX/MEM 寄存器中的 ALU 结果，以及 MEM/WB 寄存器中的 ALU 结果或存储器读出数据。
- **转发目标**：直接转发到 EX 段 ALU 的输入端口，作为当前指令的操作数。

2. **转发条件检测**：在 ID 段和 EX 段，增设转发检测逻辑。该逻辑通过比较当前指令的源寄存器编号（rs1,rs2）与前面指令的目的寄存器编号（rd），判断是否存在数据相关，并产生相应的转发控制信号。

3. **优先级**：当存在多个转发源时，离当前指令最近的结果（即 EX/MEM 段的结果）具有最高优先级，因为它是最新的数据。

4. **与气泡机制协同**：数据转发解决了绝大部分的算术逻辑运算指令的数据冒险，但对于 load-use 冲突，由于加载指令的数据在 MEM 段结束时才能获得，无法及时转发到 EX 段，因此仍然需要插入一个气泡。

通过数据转发机制，流水线避免了大多数因数据相关而产生的停顿，显著提升了性能。

对于分支冒险，重定向流水线的解决办法与气泡流水线相同。

2.6 动态分支预测机制

为减少控制冒险带来的性能损失，本设计实现了**动态分支预测**。其目标是尽可能早地预测分支指令的跳转方向和目标地址，从而让取指阶段能够不间断地获取指令。

1. **分支目标缓冲区**：在 IF 段设置一个小的缓存，其以分支指令的 PC 地址作为索引。每个表项包含：

- **预测跳转方向：**使用状态机（两位饱和计数器）实现，其状态（强不跳转、弱不跳转、弱跳转、强跳转）根据该指令历史执行结果进行动态更新。
- **预测目标地址：**记录该分支指令上一次跳转的目标地址。
- **有效位：**记录该条缓存是否有效。
- **分支指令地址：**索引
- **计时器：**实现基于 LRU（最近最少使用）近似算法的缓存替换策略

2. 预测与执行流程：

- **取指时预测：**在 IF 阶段，除了读取指令，同时用当前 PC 查询 BTB。如果命中且预测状态为“跳转”，则下一个 PC 直接采用 BTB 中存储的目标地址，实现“预测跳转”。
- **执行时验证：**当分支指令流动到 EX 段时，会计算实际的跳转条件和目标地址。
- **结果对比与更新：**
 - 若预测正确：流水线继续执行，无性能损失，同时将计时器置 0，更新状态机以强化此次预测。
 - 若预测错误：则**冲刷**掉在错误路径上取入的后续两条指令（在 IF 和 ID 段插入气泡），将 PC 更正为正确的地址，并更新 BTB 中的预测状态和目标地址。如果没有空位用于更新，则替换掉最近没有被使用的一项记录。
 - 如果 EX 阶段的跳转指令没有命中 BTB，那么也要更新。

动态分支预测机制有效降低了因分支指令造成的流水线停顿，使得流水线接近于理想流水线的执行效率。

3 详细设计与实现

3.1 单周期 CPU 实现

3.1.1 主要功能部件实现

1. 程序计数器 (PC)

程序计数器使用一个 32 位的寄存器实现，在每个时钟周期的上升沿，PC 会将当前输出的地址锁存，并将其送入指令存储器以读取当前指令。同时，通过一个专用的加法器，将 PC 当前值加 4（因为 RISC-V 指令字长为 4 字节），计算出顺序执行的下一条指令地址，与各种分支指令的跳转地址作为 PC 在下一个时钟周期的输入，使用控制信号和多路选择器来选择最终的输入。如图 3.1 PC

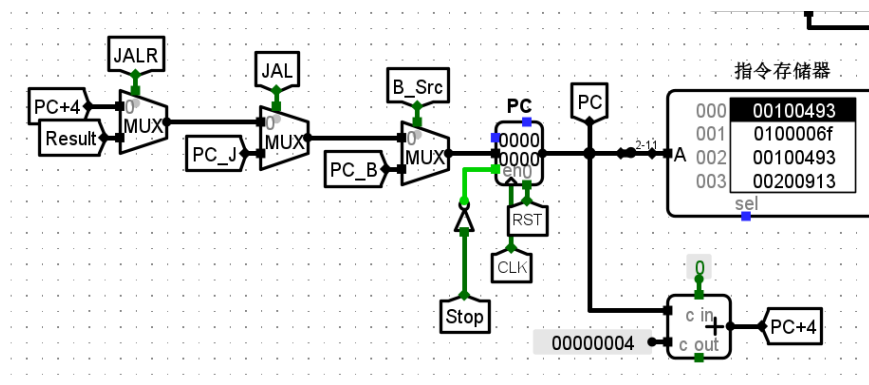


图 3.1 PC

2. 指令存储器 (IM)

使用一个只读存储器 ROM 实现指令存储器 (IM)。设置该只读存储器的地址位宽为 10 位，数据位宽为 32 位。因为 PC 中存储的指令地址有 32 位，而 ROM 地址线宽度有限，仅为 10 位，故将 32 位指令地址高位部分和字节偏移部分直接屏蔽，使用分线器只取 32 位指令地址的 2-11 位作为指令存储器的输入地址。如图 3.2 IM 所示。

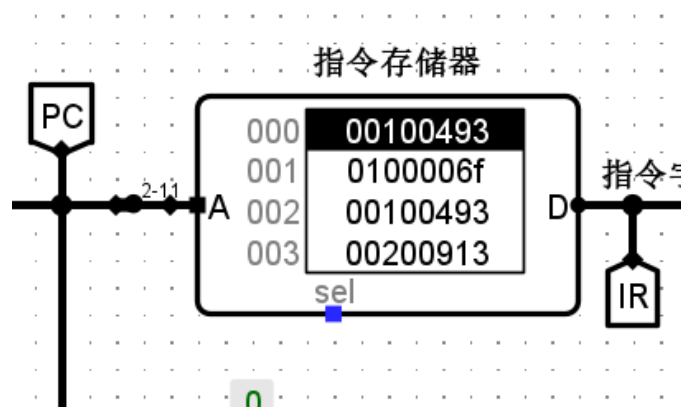


图 3.2 IM

3. 运算器

使用 U_type 信号控制运算器的 A 端输入是来自 R1 还是 0 (U 指令所需的立即数), 用 $AluSrcB$ 控制 B 端输入是来自 R2 还是立即数, 使用 $AluOP$ 控制运算的类型, 输出结果和 Equ 、 $GreEqu$ 两个信号。如图 3.3 Alu。

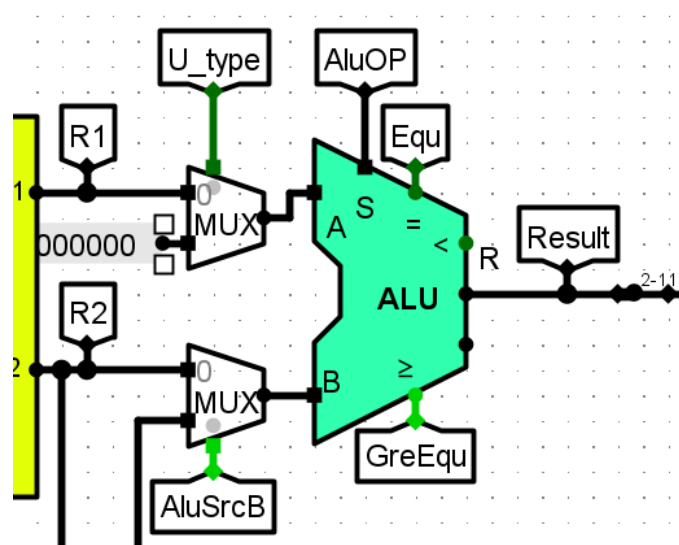


图 3.3 Alu

4. 寄存器堆 RF

$R1\#$, $R2\#$ 是要访问的寄存器编号, 默认由指令的译码给出, 当指令为 `ecall` 指令时 $R1\#$ 固定为 `0x11`, $R2\#$ 固定为 `0xa` (基于 `ecall` 指令的要求), $RegWrite$ 是写使能控制信号, $RDin$ 是写入的数据, 可能是运算器结果、数据寄存器取出的值、数据寄存器取出的值的某个字节 (`LB` 指令), 具体是哪个用控制信号选择, 输出为从两个寄存器

中读出的数据 R1, R2。如图 3.4 Reg。

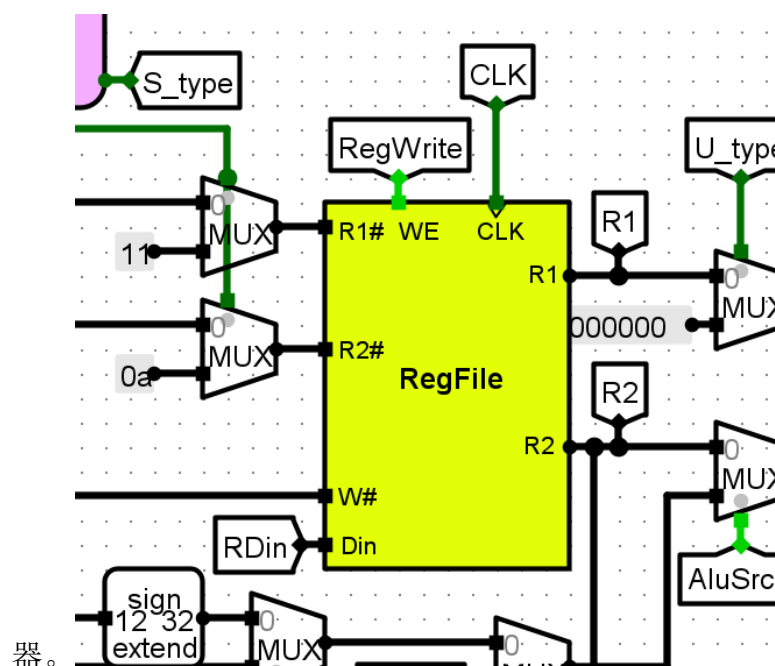


图 3.4 Reg

5. 数据存储器 DM

数据存储器地址输入是 Alu 运算结果的高 10 位(因为 DM 一个基本单元是 32bit)，数据输入是 R2，输出是读出的数据，MemWrite 是写使能信号。如图 3.5 DM。

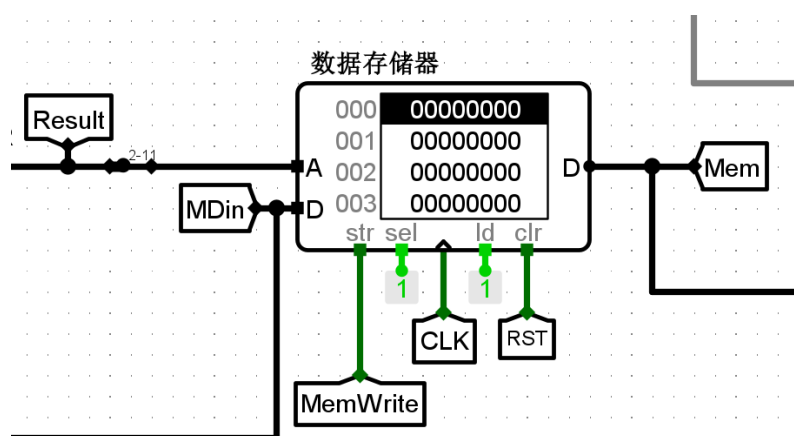


图 3.5 DM

6. 其他部件

立即数生成如图 3.6 立即数，根据指令手册的内容构建各个指令的立即数。

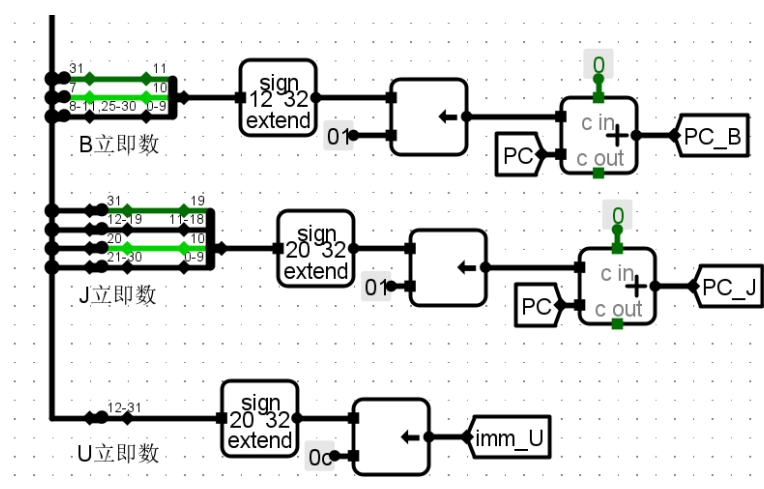


图 3.6 立即数

B 指令跳转信号生成，如图 3.7 B 跳转。

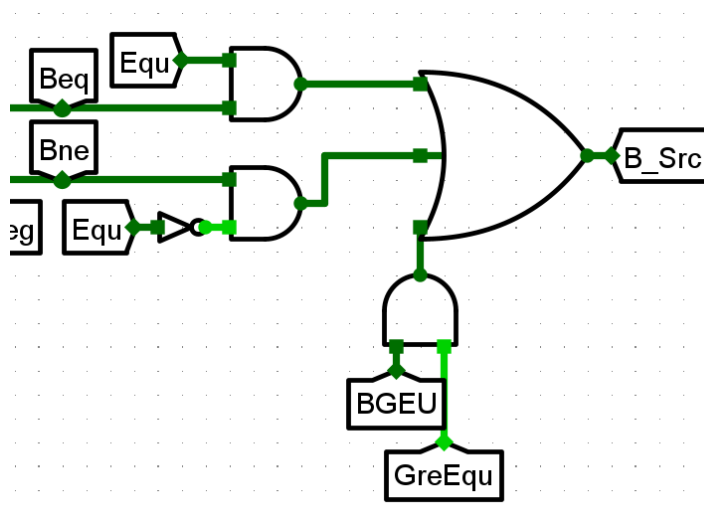


图 3.7 B 跳转

数码管显示和 ecall 指令控制暂停，如图 3.8 ecall。

华中科技大学课程设计报告

指令	PC	PC_add	IM	RF.1#	RF.2#	RF.W#	RF.D _{in}	ALU _A	ALU _B	S-EXT	DM _A	DM _{Din}	SL	PCB _{addA}	PCB _{addB}	SL2
add rd, rs, rt	PC_add	PC	PC	I[25:21]	I[20:16]	I[15:11]	ALU _{Re}	RF _{RD1}	RF _{RD2}							
slt rd, rs, rt	PC_add	PC	PC	I[25:21]	I[20:16]	I[15:11]	1:ALU _{Re,rs} =1 0:ALU _{Re,rs} =0									
Addi rt,rs,imm	PC_add	PC	PC	I[25:21]		I[20:16]	ALU _{Re}	RF _{RD1}	S-EXT	I[15:0]						
lw rt, imm(rs)	PC_add	PC	PC	I[25:21]		I[20:16]	DM	RF _{RD1}	S-EXT	I[15:0]	ALU _{Re}					
Sw rt, imm(rs)	PC_add	PC	PC	I[25:21]	I[20:16]			RF _{RD1}	S-EXT	I[15:0]	ALU _{Re}	RF _{RD2}				
beq rs, rt, imm	PC_add PCB_add	PC	PC	I[25:21]	I[20:16]			RF _{RD1}	RF _{RD2}	I[15:0]			S-EXT	SL	PC_add	
j addr	pc_add SL2*															I[25:0]

图 3.10 控制信号确认

但是本次课程设计的指导已经给出了需要实现的控制信号，只需要考虑额外的差异化指令需要的控制信号即可。如表 3.1 差异化指令控制信号所示，我们只需要添加 U_type，Byte，BEGU 三个控制信号即可。

表 3.1 差异化指令控制信号

指令	类型	作用	控制信号
SRA	R	将寄存器中的值视为有符号整数（补码表示），并向右移动指定的位数。空出的高位用原数的符号位（即最高位）来填充。	使用现有的数据通路，无需额外控制信号
LUI	U	将一个 20 位的立即数加载到目标寄存器的高 20 位，并将目标寄存器的低 12 位置零。	首先需要控制信号控制立即数输入到 Alu 的 B 端口，其次我选择在立即数构造时就完成左移 12 位，后续只需要在 Alu 中与 0 按位异或即可，所以总共需要一个 U_type 信号
LB	I	从内存中加载一个字节（8 位）的数据	由于其他的读取内存指令都只读出 32

BGEU

B

到目标寄存器，并进位数据，所以需要
行符号扩展到寄存一个 Byte 信号控制
器的位宽。字节数据流向寄存
器堆

比较两个寄存器中
的值，将它们视为无
符号整数。如需要相应的控制信
果 rs1 >= rs2, 则进行号 BGEU 参与是否
程序跳转(分支); 否跳转的判断
则，顺序执行下一条
指令。

我们将各个指令所需的控制信号填入实验资料包中的“单周期硬布线控制器表达式自动生成.xlsx”（如图 3.11 控制信号表达式生成），得到各个控制信号的逻辑表达式，之后再利用 logisim 的电路分析功能自动生成“运算控制器自动生成”和“控制信号自动生成”两个模块，将这两个模块简单集成就能构建出控制器（如图 3.12 控制器实现）。

#	指令	Func17 (十进制)	Func13 (十进制)	OpCode (十六进制)	ALU_OP	MemtoReg	MemWrite	ALU_Src	RegWrite	ecall	S_Type	BEQ	BNE	Jal	Jalr	CSRRSI	U_type	Byte	BGEU	CSRRCI
1	add	0	0	C	5				1											
2	sub	32	0	C	6				1											
3	and	0	7	C	7				1											
4	or	0	6	C	8				1											
5	slt	0	2	C	11				1											
6	sltu	0	3	C	12				1											
7	addi	0	4	4	5			1	1											
8	andi	7	4	7	7			1	1											
9	ori	6	4	8	8			1	1											
10	xori	4	4	9	9			1	1											
11	shti	2	4	11	11			1	1											
12	slli	0	1	4	0			1	1											
13	srlr	0	5	4	2			1	1											
14	srai	32	5	4	1			1	1											
15	lw	2	0	5	1	1		1	1											
16	sw	2	8	5	5		1	1	1		1									
17	ecall	0	0	1C						1										
18	beq	0	18	6								1								
19	bne	1	18	6									1							
20	jal		1B						1					1						
21	jalr	0	19	6				1	1						1					
22	CSRRSI	6	1C	8				1								1				
23	CSRRCI	7	1C	7				1												1
24	URET	2	0	1C					1											
25	sra	32	5	C	1				1											
26	lui		D	8				1	1								1			
27	lb	0	0	5	1	1		1	1									1		
28	bgeu	7	1B	12															1	

图 3.11 控制信号表达式生成

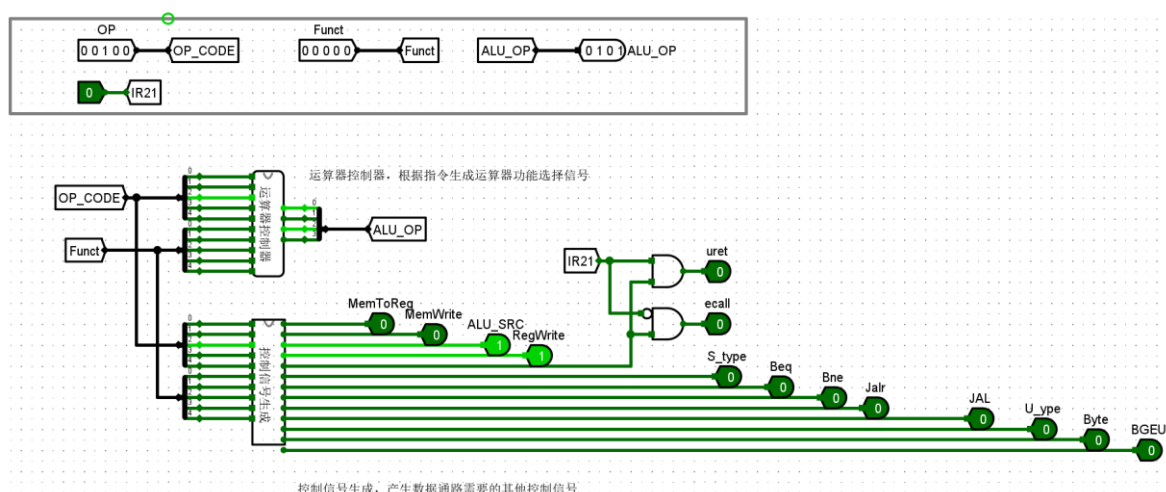


图 3.12 控制器实现

3.2 中断机制实现

3.2.1 中断按键信号采样与中断向量表

如图 3.13 中断按键信号采样与中断向量表所示，IR 就是中断请求寄存器，存储中断请求信号，输出送往中断优先编码器得到应当优先处理的中断号和中断信号，Clear 同步清零信号用于中断服务程序执行到中断返回时清除中断请求信号，W 是中断等待指示 LED 的控制信号，用于指示当前中断请求，在中断服务程序返回时熄灭。

3 个中断服务程序程序入口地址被硬编码进硬件里，利用多路选择器和中断号选择出来当前要执行的中断服务程序程序入口地址。

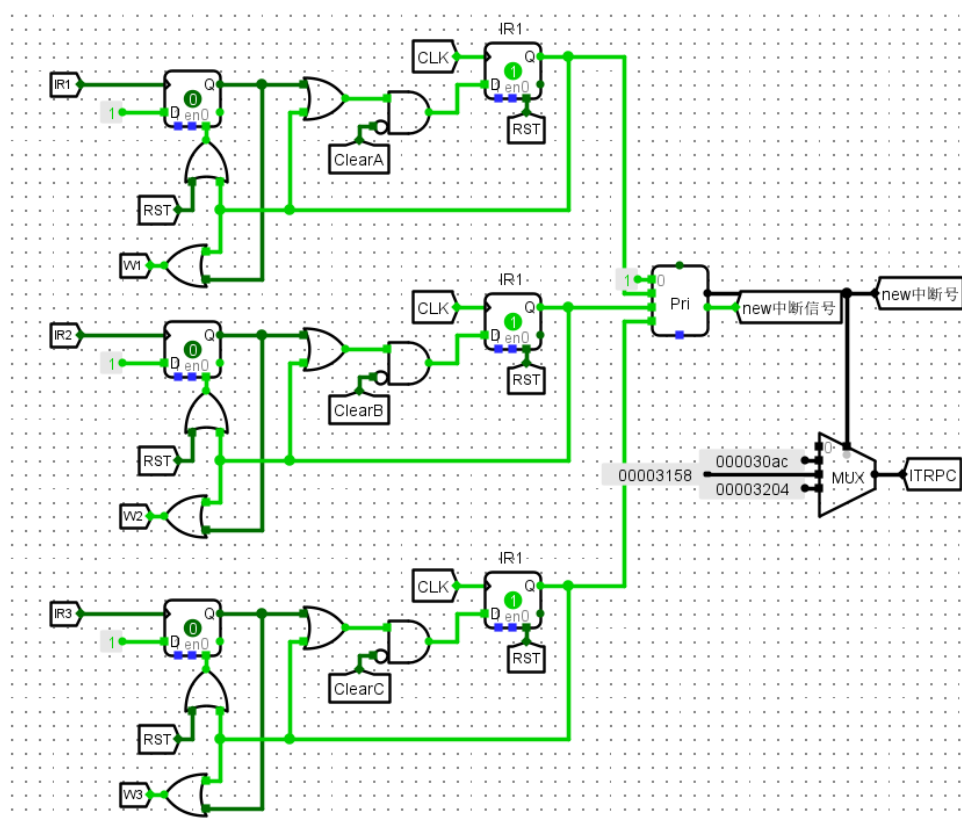


图 3.13 中断按键信号采样与中断向量表

3.2.2 中断优先级的确定

如图 3.14 中断优先级所示，我们借助 3 个寄存器记录了当前正在处理的中断号（pre 中断号），当产生了新的中断信号时，如果新中断号大于正在处理的中断号就允许打断当前中断。

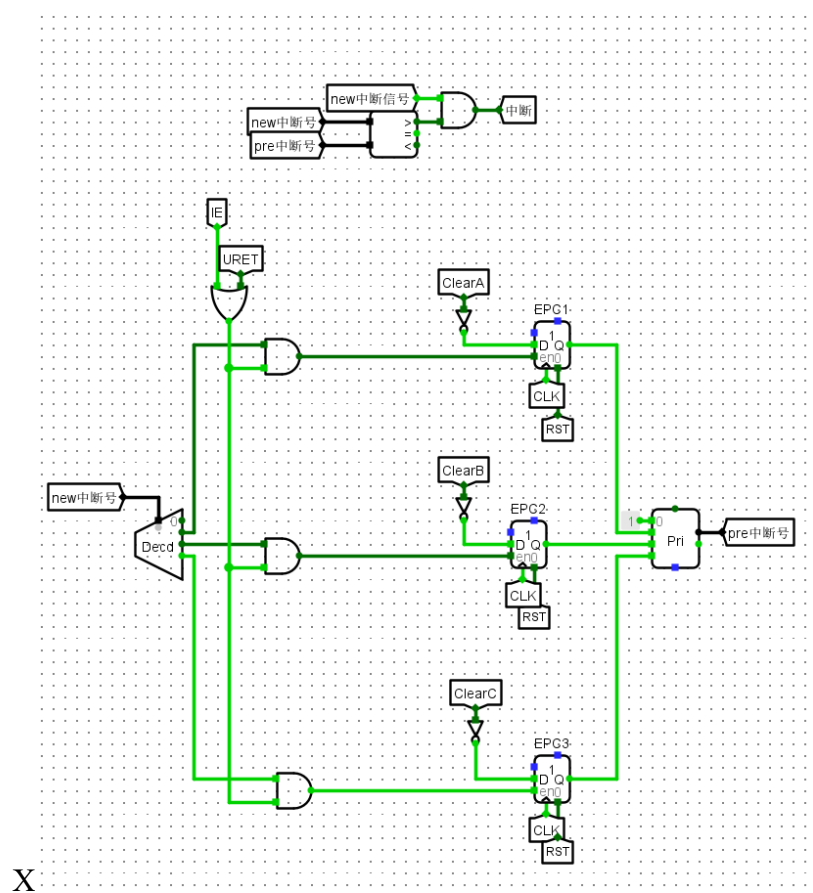


图 3.14 中断优先级

3.2.3 中断使能寄存器

如图 3.15 中断使能寄存器所示，中断使能寄存器 IE 用于开关中断，1 表示开中断，0 表示关中断。

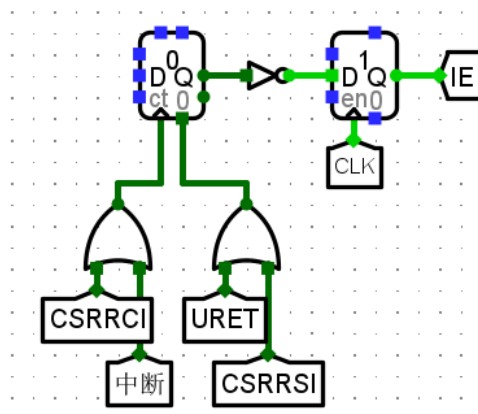


图 3.15 中断使能寄存器

图 3.16 硬件堆栈保护 EPC 寄存器

3.2.5 中断返回

图 3.17 中断返回

图 3.18 跳转中断服务程序入口

3.3 流水 CPU 实现

3.3.1 流水接口部件实现

每个流水接口都用一组寄存器,在时钟上升沿锁存并传递前一阶段的所有输出信息,为下一阶段提供稳定的输入。此外有清零和暂停信号以支持分支冒险和数据冒险的处理。如图 3.19 流水接口部件示例所示。

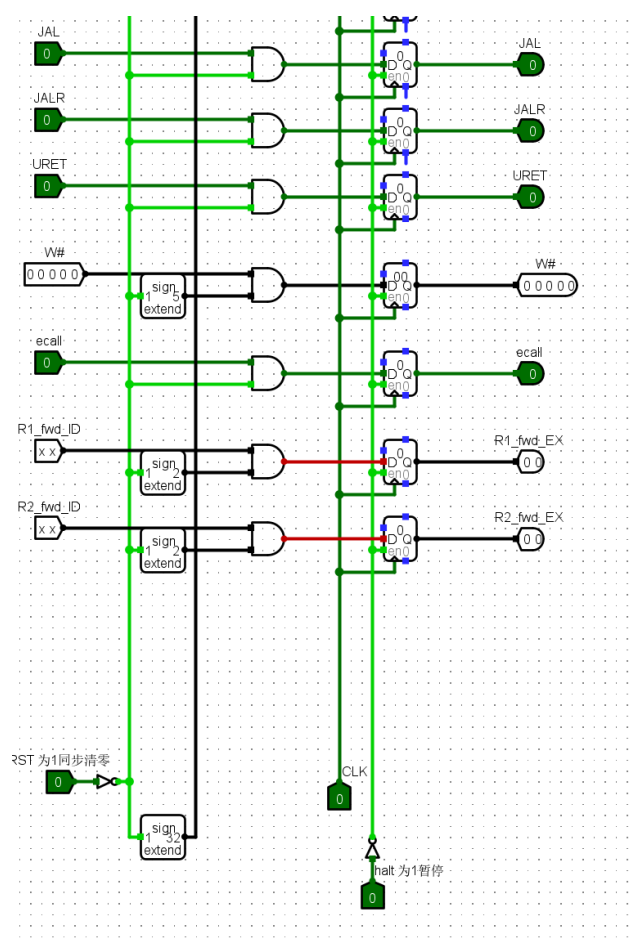


图 3.19 流水接口部件示例

3.3.2 理想流水线实现

如图 3.20 理想流水线所示，我们只需要简单地将单周期 CPU 分为 5 个部分，在每个部分之间用我们构建的流水接口部件连接起来即可。

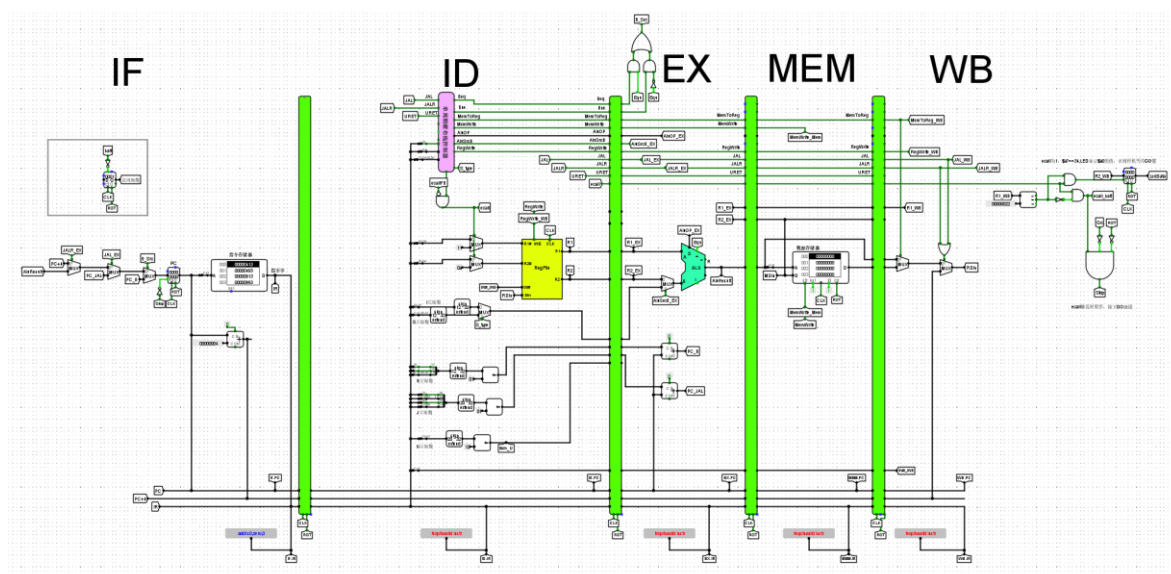


图 3.20 理想流水线

3.4 气泡式流水线实现

气泡流水线只需要在理想流水线的基础上解决分支冒险和数据冒险即可。

分支冒险的解决很简单，我们只需要构建一个 Branch 信号，当 EX 段有跳转指令需要跳转时，就利用 Branch 信号让 IF/ID 和 ID/EX 插入气泡即可。

为了解决数据冒险，我们需要构建数据冒险的检测电路（图 3.21 数据冒险）。图中右上角的部件是检测当前指令是否使用了 R1 或者 R2，可以借助“单周期硬布线控制器表达式自动生成.xlsx”（图 3.11 控制信号表达式生成）和 logisim 的电路分析功能构建，当 ID 段的 R1#或 R2#与 EX 或 MEM 段的 W#相同，EX 段、MEM 段有寄存器堆的写使能控制信号，并且 ID 段的指令使用了 R1 或者 R2，我们就判定为数据冒险，暂停 IF/ID，给 ID/EX 插入气泡。

最终 CPU 实现如图 3.22 气泡流水线。

华中科技大学课程设计报告

定向装置的输出信号和多路选择器完成重定向（图 3.25 重定向）。

最终 CPU 实现如图 3.26 重定向流水线。

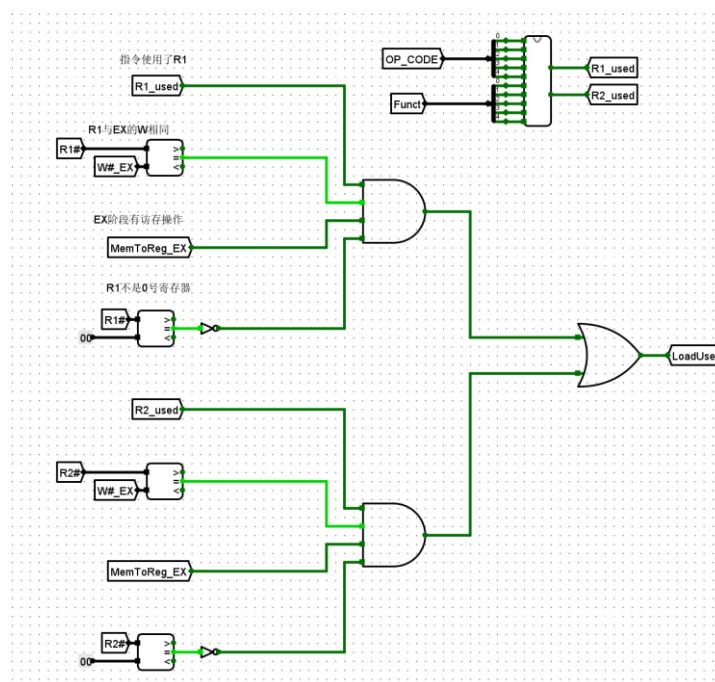


图 3.23 LoadUse 检测器

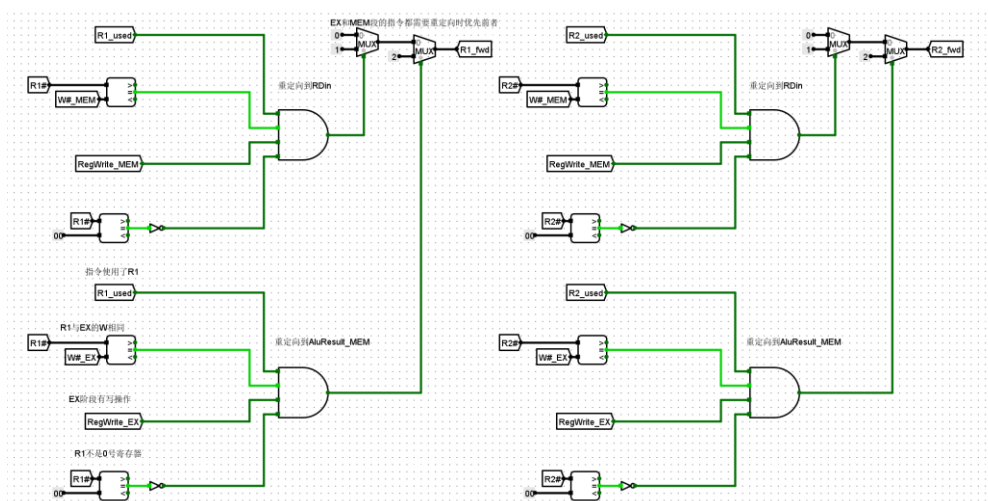


图 3.24 重定向装置

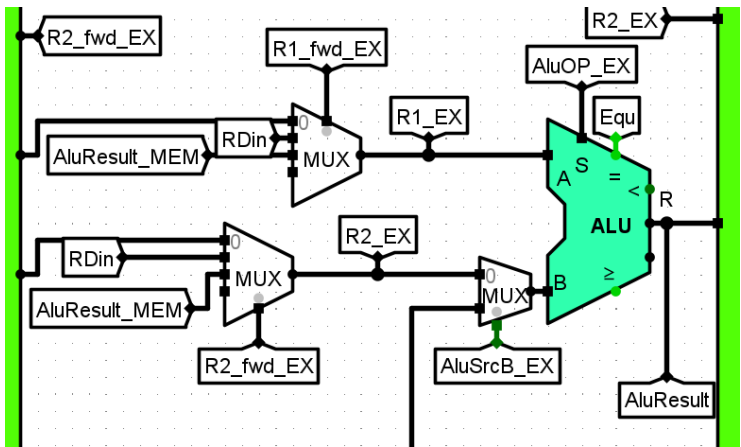


图 3.25 重定向

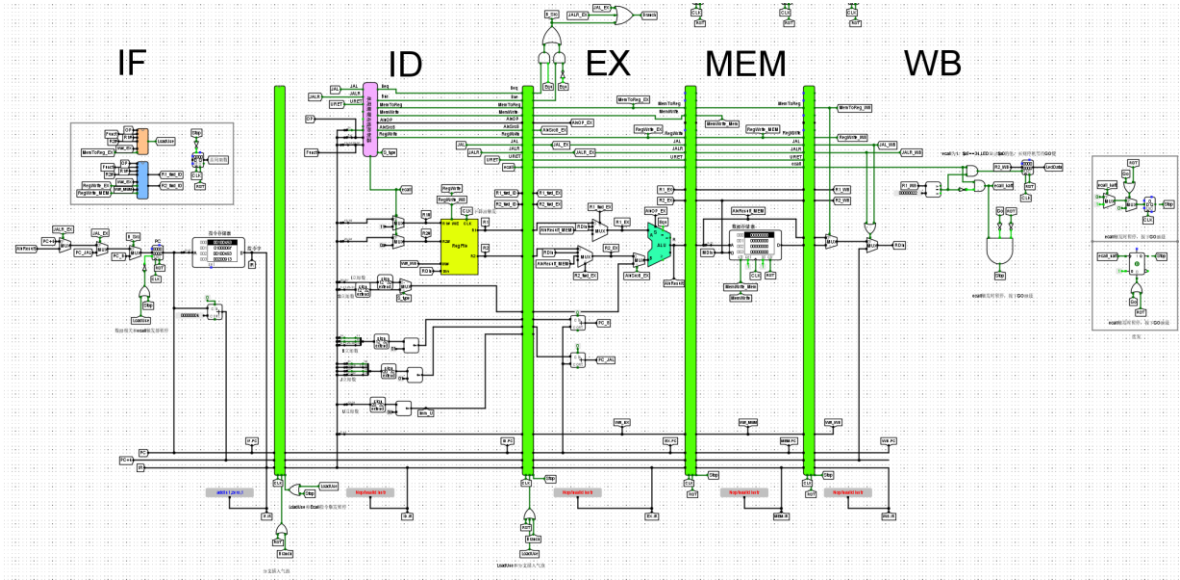


图 3.26 重定向流水线

3.6 动态分支预测机制实现

1. BTB 分支目标缓冲区

如图 3.27 BTB 所示缓冲区共有 8 行，每一行有 4 个寄存器和一个计数器，作用如表 3.2 缓冲区部件所示。

表 3.2 缓冲区部件

部件	作用
有效位寄存器	表示该条缓存是否有效。
分支指令地址	IF 段查表预测时的索引

华中科技大学课程设计报告

记录该分支指令上一次跳转的目标地址。
预测目标地址

共有 4 种状态（强不跳转、弱不跳转、
弱跳转、强跳转）
状态机

实现基于 LRU（最近最少使用）近似算
法的缓存替换策略
计时器

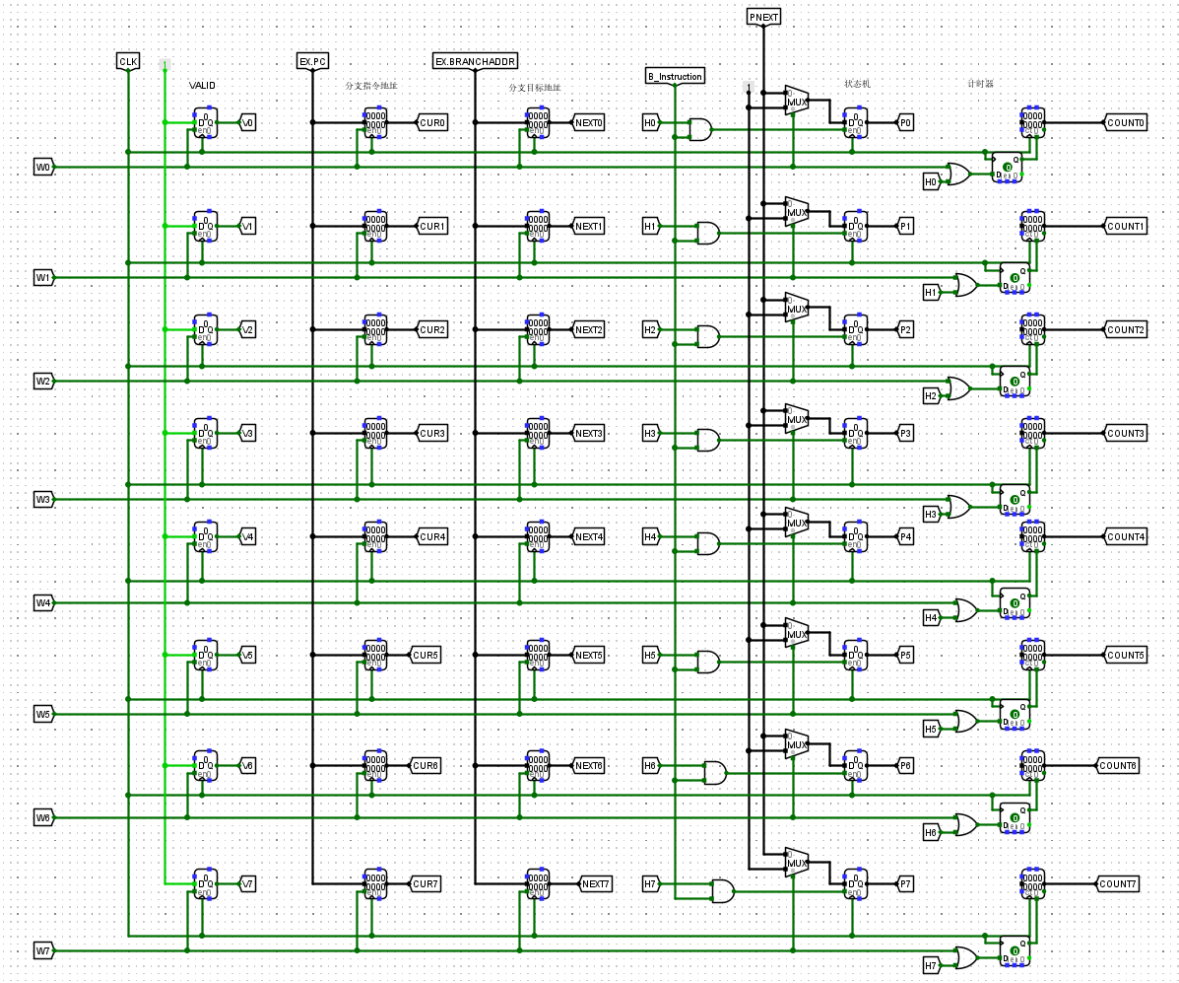


图 3.27 BTB

2. IF 预测逻辑

如图 3.28 预测逻辑所示，当 IF 段的 PC 查表成功，且状态机状态为跳转，那么就取出预测目标地址用于跳转。

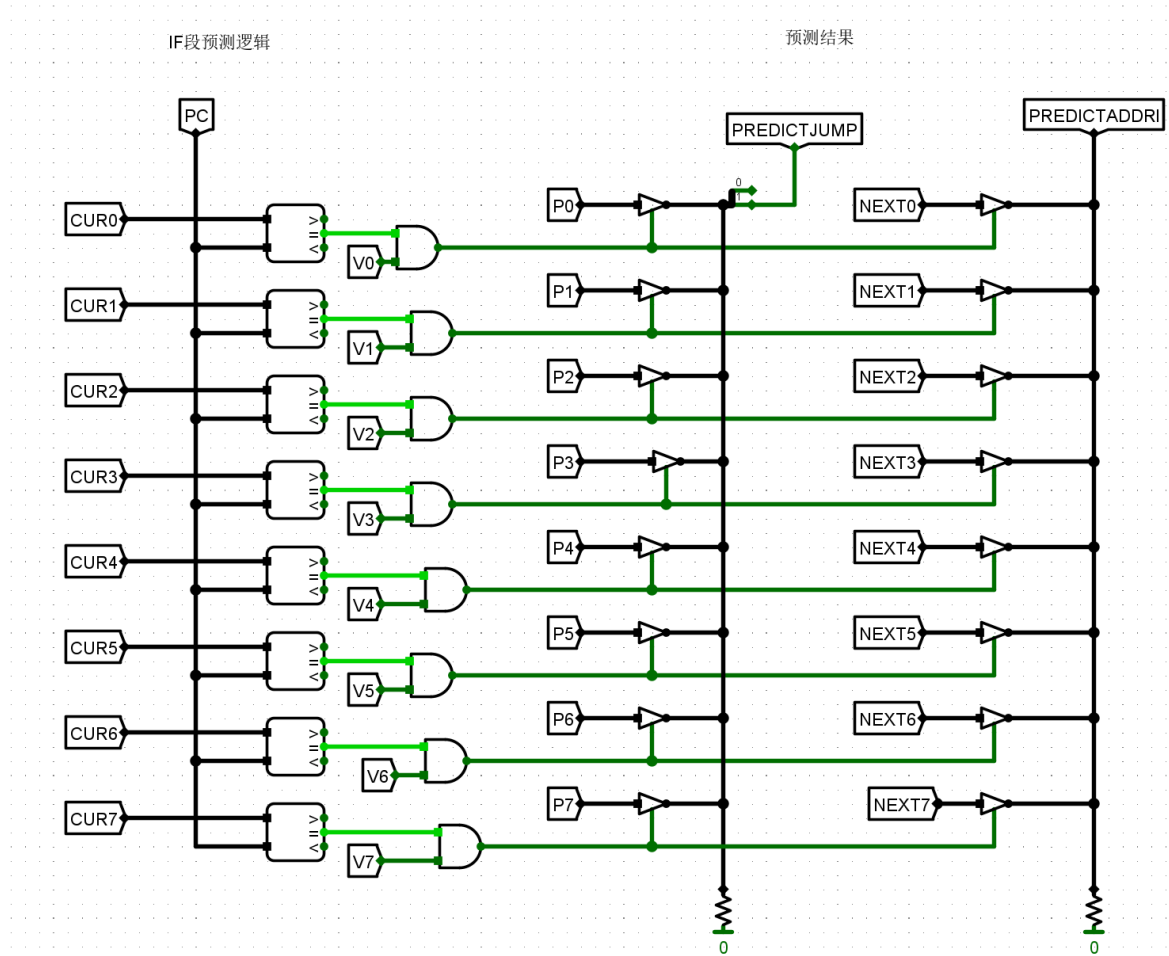


图 3.28 预测逻辑

3. EX 检验逻辑

如图 3.29 EX 检验逻辑所示，如果 EX 阶段发现此时是跳转指令但没有命中，那么就选择一个有效位为 0 的位置更新缓存，如果有效位全为 1 就选计时器最大的替换掉。

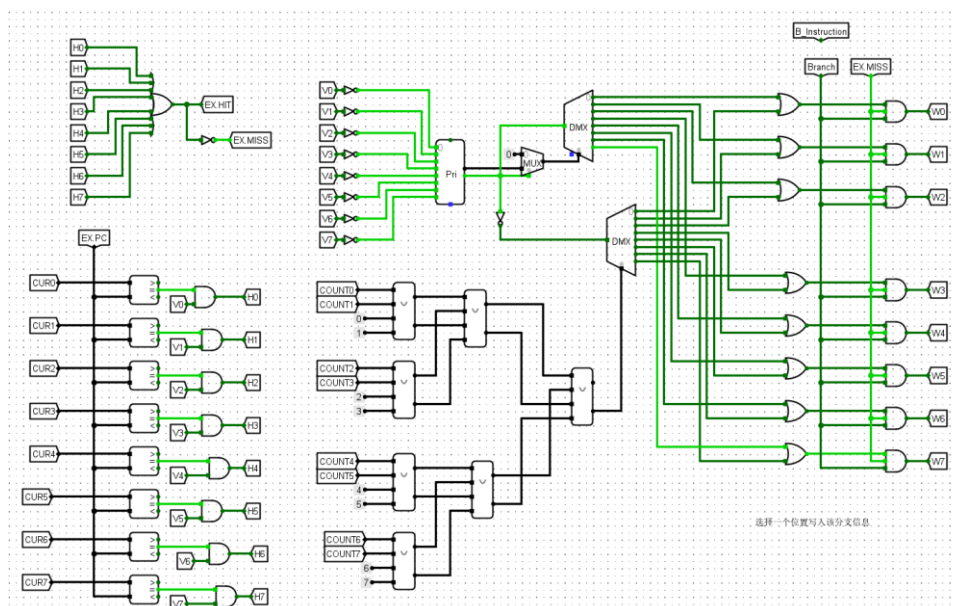


图 3.29 EX 检验逻辑

4. 状态机实现

如图 3.30 状态图所示，00 代表强不跳转，01 代表弱不跳转，10 代表弱跳转，11 代表强跳转，绿线代表分支指令跳转，红线代表分支指令不跳转。每一条新缓存状态初值都设为 01。状态机实现电路如图 3.31 状态机所示。

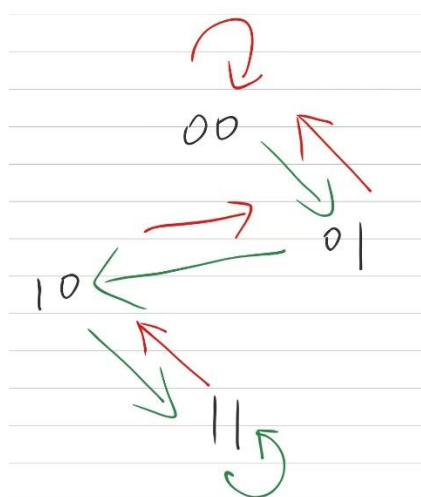


图 3.30 状态图

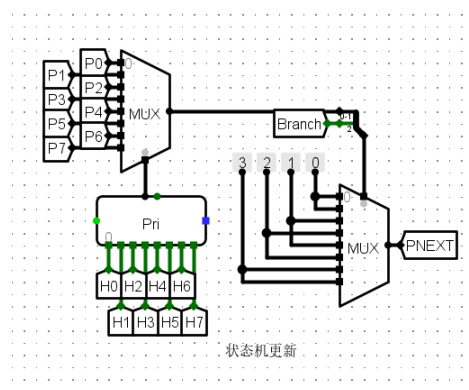


图 3.31 状态机

4 实验过程与调试

4.1 测试用例和功能测试

4.1.1 头歌平台测试

头歌平台提供了对本次实验中的单周期 CPU（24 条指令）、理想流水线、气泡流水线、重定向流水线、单周期 CPU 单级中断、单周期 CPU 多级嵌套中断（EPC 硬件堆栈保存）的测试（表 4.1 头歌测试），测试结果全部通过（图 4.1 头歌测试），本地运行也符合逻辑。

表 4.1 头歌测试

测试	用例
单周期 CPU（24 条指令）	risc-v-benchmark.hex
理想流水线	risc-v-理想流水线测试.hex
气泡流水线	risc-v-benchmark.hex
重定向流水线	risc-v-benchmark.hex
单周期 CPU 单级中断	risc-v 单级中断测试程序.hex
单周期 CPU 多级嵌套中断（EPC 硬件堆栈保存）	risc-v 多级中断测试(EPC 硬件堆栈保护).hex

华中科技大学课程设计报告

序号	关卡名称	关卡分值	我的得分	通过状态	耗时	评测次数	难度	通过率	知识点
1	RISC-V单周期CPU(24条指令)	11.11	11.11	已通过	1时 6分 20秒	9	困难	95.05%	RISC-V单周期21条
2	理想流水线设计	11.11	11.11	已通过	2天 1时 15...	7	中等	94.12%	RISC-V理想流水线
3	气泡流水线设计(EX段分支3624版本)	11.11	11.11	已通过	45分 0秒	10	困难	91.64%	RISC-V气泡流水线
4	重定向流水线(EX段分支2298版本)	11.11	11.11	已通过	23分 44秒	14	困难	91.95%	RISC-V重定向
5	重定向流水线设计(ID段分支2103版...	11.11	0.00	未评测	--	0	困难	1.86%	RISC-V重定向2103
6	气泡流水线设计(ID段分支3309版本)	11.11	0.00	未评测	--	0	困难	1.86%	RISC-V气泡流水线3309
7	单周期RISC-V+单级中断	11.11	11.11	已通过	9分 58秒	10	困难	91.02%	RISC-V单级中断
8	多级嵌套中断(EPC硬件堆栈保存)	11.11	11.11	已通过	2时 12分 49...	8	困难	86.38%	RISC-V多级中断
9	多级嵌套中断(EPC内存堆栈保存)	11.12	0.00	未评测	--	0	困难	0.31%	RISC-V多级中断堆栈方式

图 4.1 头歌测试

4.1.2 单周期 CPU（24 条指令+差异化指令）

这里我们将需要实现的 4 个差异化指令的测试程序粘贴到 risc-v-benchmark_ccab.asm 程序中的相应位置（图 4.2 差异化指令测试程序），使用 RARS 软件汇编为 16 进制的机器码导入指令存储器运行，每当程序停止时按下继续按钮，LED 灯就会输出 4 条指令的测试程序规定的输出，测试结果不方便图文展示，此处不再给出。

```
ecall ..... # we are out of here. ....

#####
# insert your ccmb benchmark program here!!!

#sra移位测试 .. revise date:2021/1/24 tiger
#依次输出: 0x87600000 0xf8760000 0xffff87600 0xffff8760 0xffff876 0xffff87 0xffff87 0xffff87 0xffff87 0xffff87
#LUI测试 .. revise date:2021/1/24 tiger
#依次输出: 0xfedcffff 0x0ba98000 0x07654000 0x03210000 0xfedcffff 0x0ba98000 0x07654000 0x03210000 0xfedcffff 0x0ba9
#LB测试 .. revise date:2021/1/24 tiger
#依次输出: 0xffffffff81 0xffffffff82 0xffffffff83 0xffffffff84 0xffffffff85 0xffffffff86 0xffffffff87 0xffffffff88 0xffffffff89 0xffffffff8a 0xffffffff8b 0xffffffff8c 0xffffffff8d 0xffffffff
#bgeu 测试 .. 大于跳转 .. 递减运算 .. 从正数开始向零运算 .. revise date:2022/2/24 tiger
#依次输出:0x0000000f 0x0000000e 0x0000000d 0x0000000c 0x0000000b 0x0000000a 0x00000009 0x00000008 0x00000007 0x00000006 0x00000005 0x00000004 0x00000003 0x00000002 0x00000001 0x00000000

#####

#j benchmark_start ..... #delete this instruction for ccmb benchmark
#C1 instruction benchmark

#sra移位测试 .. revise date:2021/1/24 tiger
#依次输出: 0x87600000 0xf8760000 0xffff87600 0xffff8760 0xffff876 0xffff87 0xffff87 0xffff87 0xffff87 0xffff87

.text

addi t0,zero,1 ..... #sll 移位次数
addi t1,zero,3 ..... #sll 移位次数
addi s1,zero, 0x8
slli s1,s1,8
addi s1,s1,0x76 .....

slli s1,s1,20 ..... #

add a0,zero,s1 .....
addi a7,zero,34 ..... # system call for print
ecall ..... # print
```

图 4.2 差异化指令测试程序

华中科技大学课程设计报告

4.1.3 流水线单极中断测试

把 risc-v 单级中断测试程序.hex 加载到指令存储器运行，一次点击中断 1，2，3，CPU 依次进入并返回中断 1，3，2 的中断服务程序，测试成功。

4.1.4 动态分支预测测试

载入 risc-v-benchmark.hex 运行 CPU，LED 灯显示正常，最后的周期数也合理，运行结果如图 4.3 动态分支预测测试。

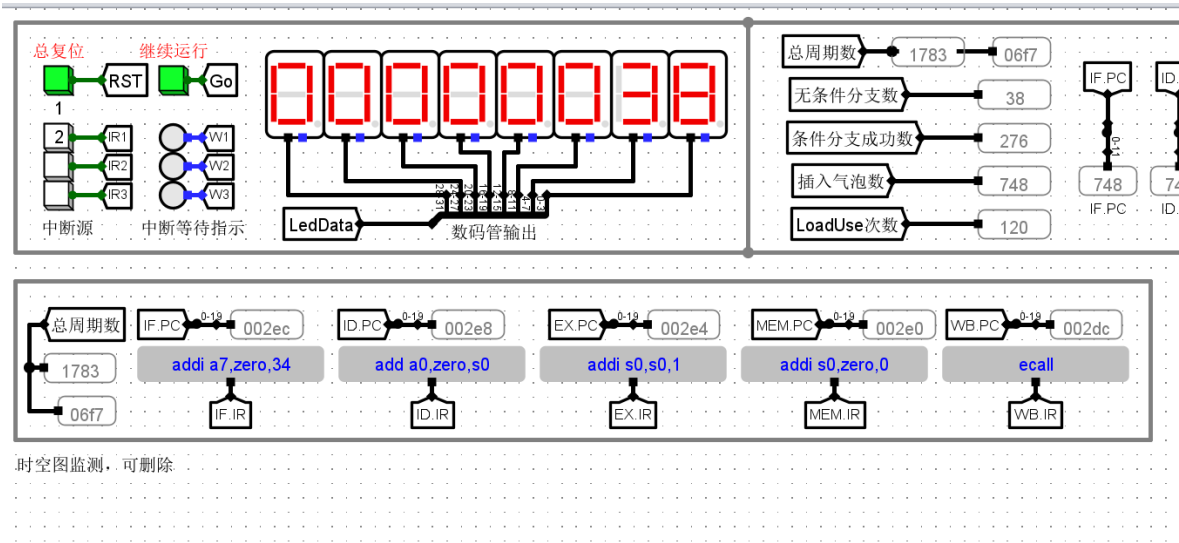


图 4.3 动态分支预测测试

4.2 性能分析

气泡流水线 CPU、重定向流水线 CPU、动态分支预测 CPU 的时钟周期数差异如表 4.2 性能对比所示。可见这三种 CPU 的运行周期逐渐降低，性能依次提高，符合预期。

表 4.2 性能对比

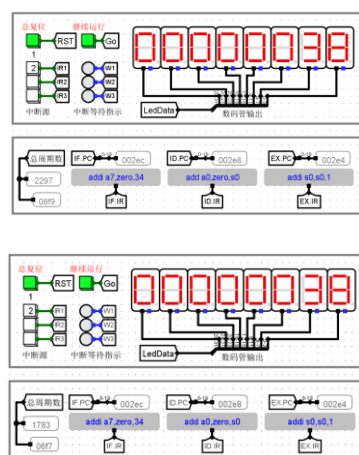
CPU	周期数	图片
气泡流水线	3623	

重定向流水线

2297

动态分支预测

1738



4.3 主要故障与调试

4.3.1 故障一

程序停不下来，单步执行发现 jal 指令寄存器输入 RDin 错误（图 4.4 故障一），应当为 RDin 添加 PC 加 4 的输入来源，并在指令为 jal 或 jalr 时使用这个来源。

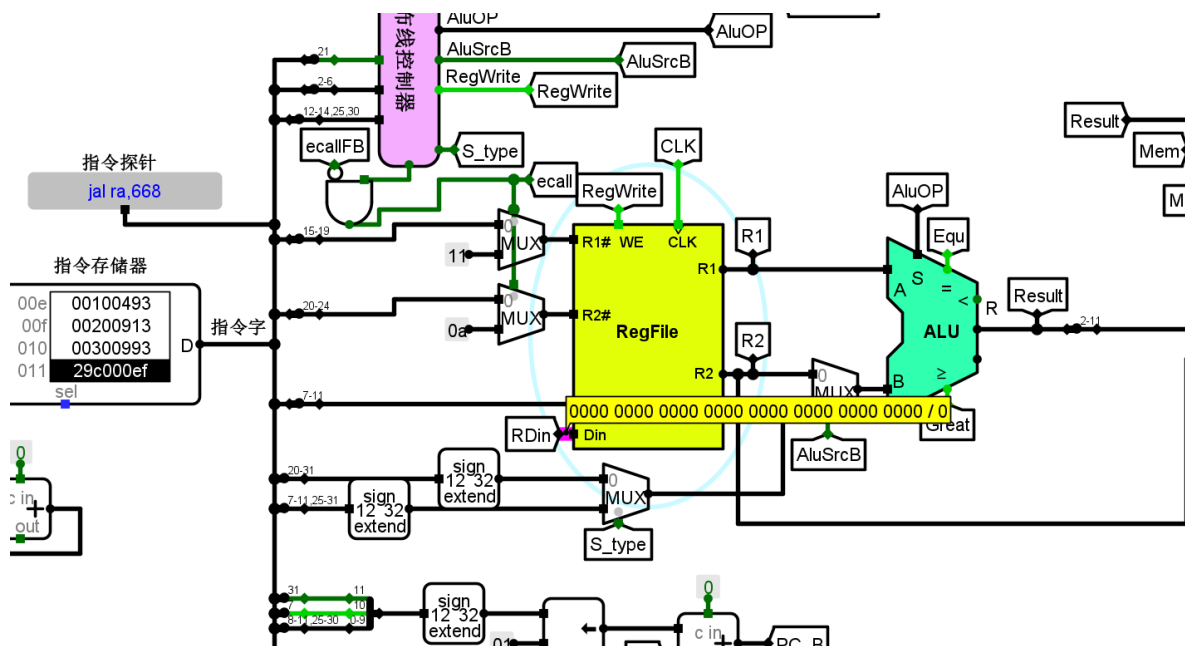


图 4.4 故障一

4.3.2 故障二

单周期支持 ccab 的 LUI 指令时信号灯与预期输出不符，检查 LUI 的数据通路，

华中科技大学课程设计报告

发现立即数要左移 12 位，但是应该用 16 进制表示，我把 0xc 误写为 0x12（图 4.5 故障二）。

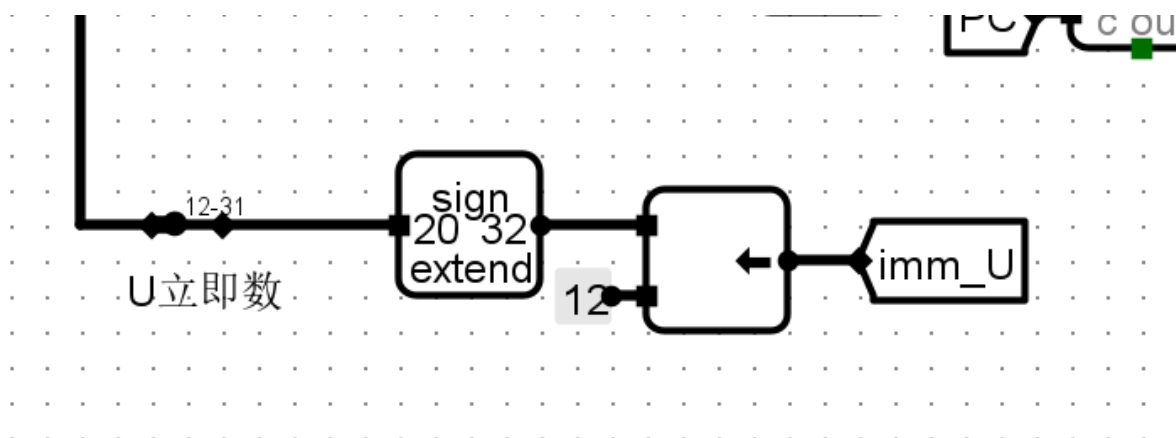


图 4.5 故障二

4.3.3 故障三

流水线复位按钮失效，没有注意到模板中自带复位按钮，自己又添加了一个，删掉即可。

4.3.4 故障四

流水线运行出错，发现流水接口部件失效(图 4.6 故障四)，应该是 logisim 出错，重启解决。

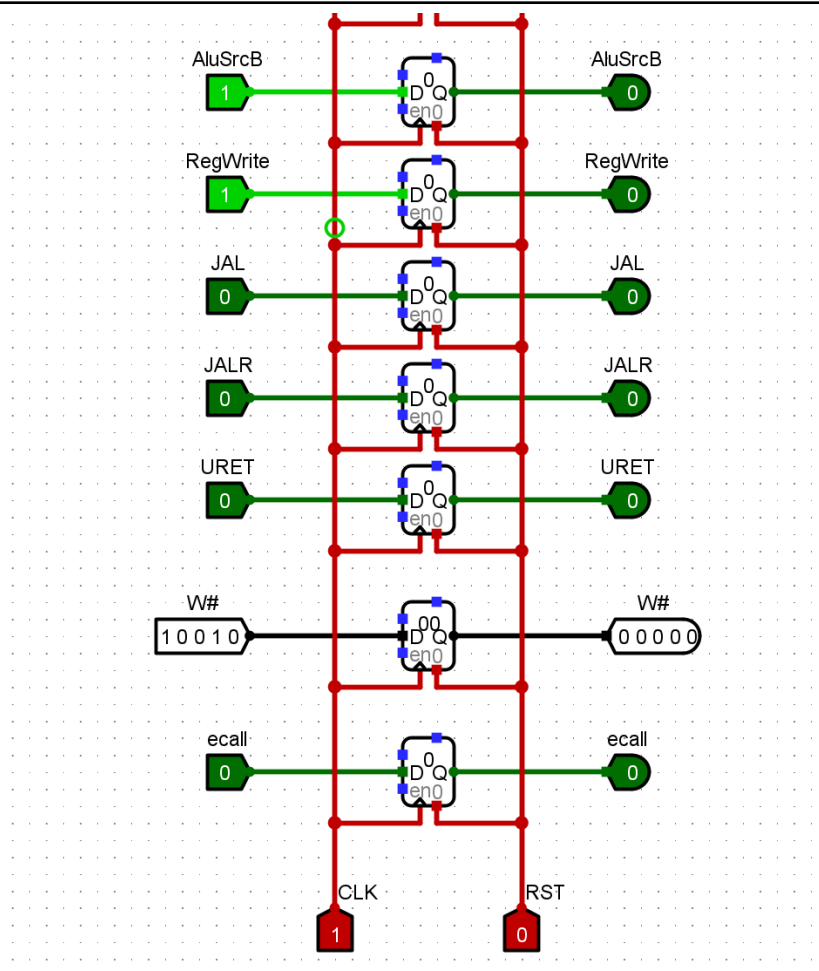


图 4.6 故障四

4.3.5 故障五

节拍 3 时 RegWrite 错误，检查发现时空指令的控制信号错误，RegWrite 应为 0 实为 1，改正控制器即可。

4.3.6 故障六

气泡流水线运行到第 30 个时钟周期时 ecall 指令时意外暂停，但 \$a7! = 34\$，单步调试发现第 29 个时钟周期时 ecall_halt 的与门下侧输入为 1，上侧输入为 0，而下一时刻与门的上侧输入为 1，下侧输入为 0，由于信号的时延不同，使用的 D 寄存器是异步的，所以存在竞争和险象，导致有一瞬间 ecall_halt 信号为 1，寄存器捕捉到了这个错误的信号，第一次尝试使用图中右侧的同步方案解决（图 4.7 故障六.1）。

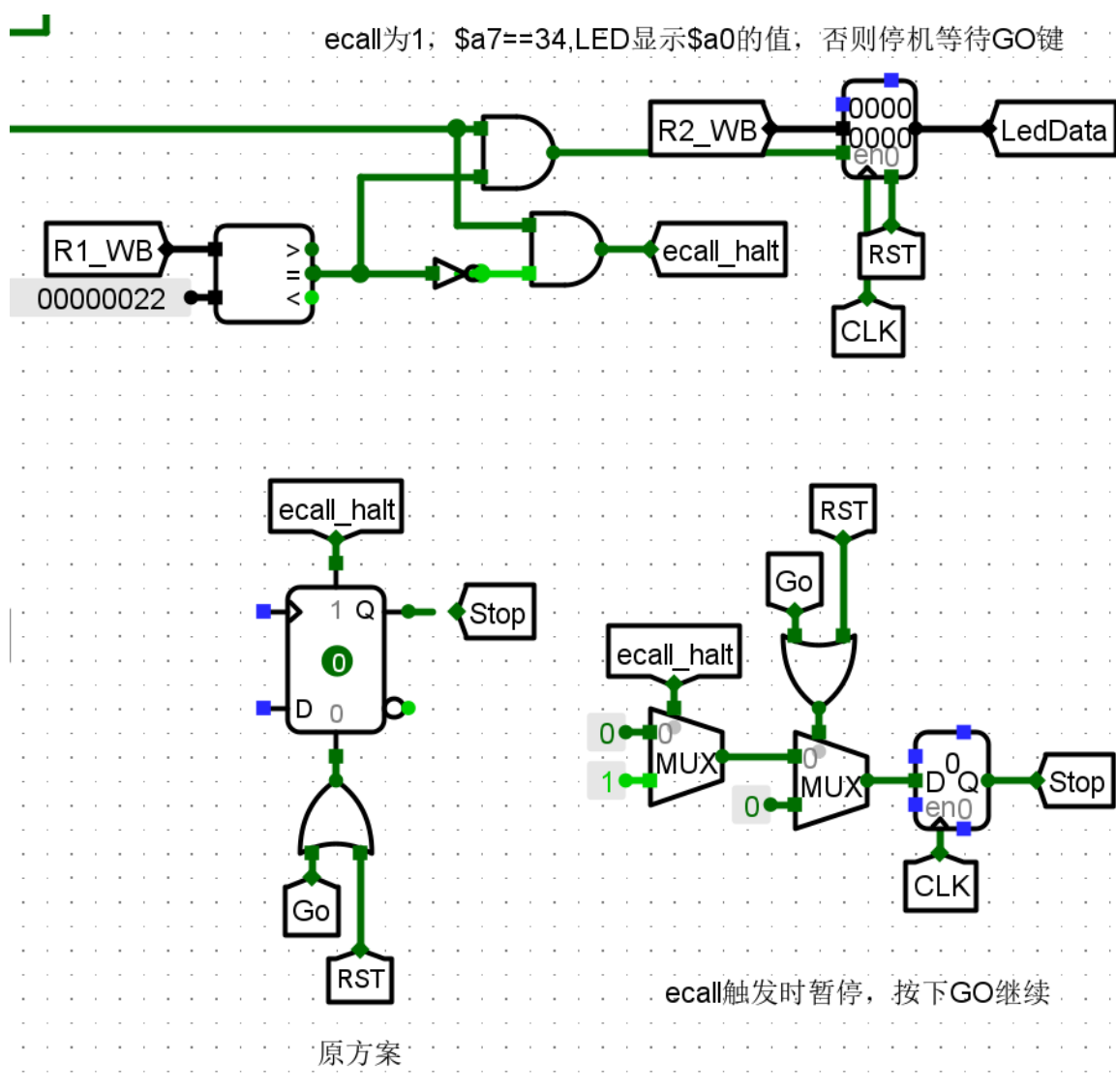


图 4.7 故障六.1

但是出现了新的问题：真正需要 ecall 暂停的最后一步缺暂停不了了，因为触发暂停时因为寄存器下一个周期才能变为 1，也就是下一个周期才会暂停，而这会使所有指令向前一步，这样 WB 段就不是 ecall 指令了，下下个周期 stop 信号会变回 0 导致 CPU 继续运行。所以我们需要一个办法，使得暂停信号既能避免险象还能在需要暂停时立刻更新。其实也完全不同步，不借助任何存储设备只需要一个与门就能实现按下 GO 继续运行的功能，虽然还会有险象，但由于 PC 的更新和流水线的传递都需要借助时钟信号，所以不在时钟信号更新时刻的险象会失效（图 4.8 故障六.2）。

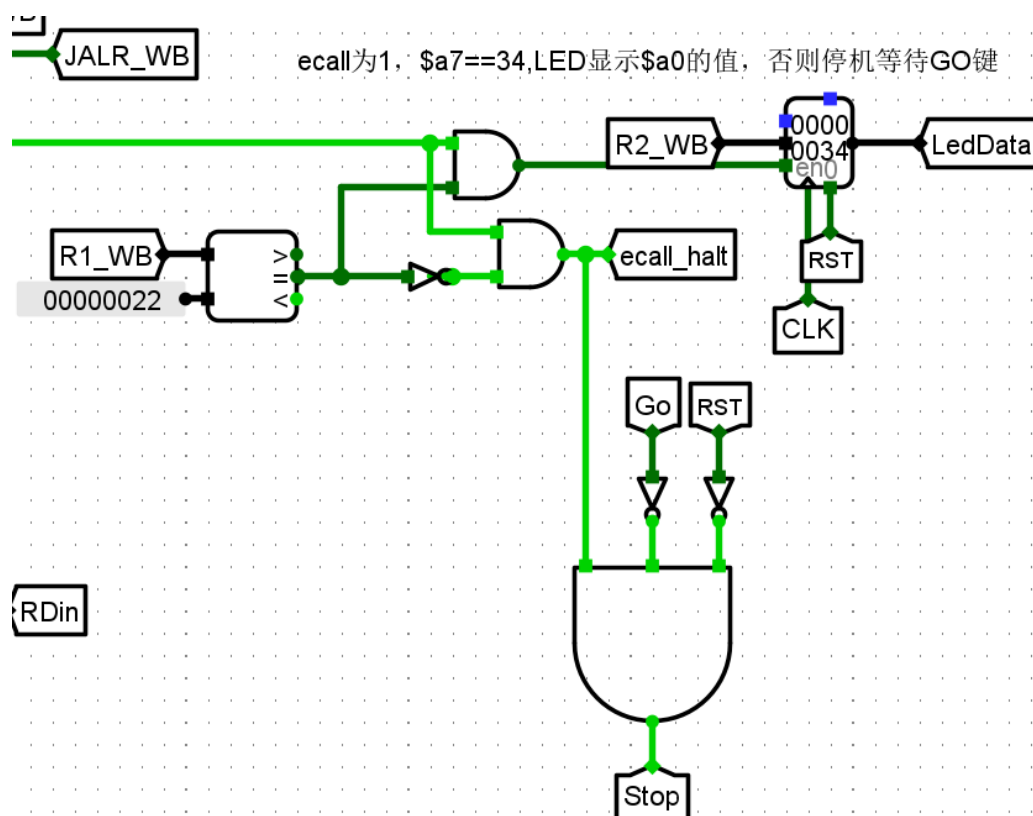


图 4.8 故障六.2

4.3.7 故障七

重定向流水线 0x16 时钟周期时, RDin 应该为 1 但实际上为 0(图 4.9 故障七.1, 图 4.10 故障七.2), 此时是 `add a0, zero, s0`。

指令运行到 WB 段, 观察前两条指令, 都需要重定向, 此时应该将最近的一条指令重定向, 运行完前两条指令 s0 为 1 正好印证了 RDin 应该为 1, 但实际上为 0, 说明重定向失败。观察电路, 意识到重定向的 forward 信号应该在 ID 段产生, 然后经过流水线的寄存器, 在下一个周期出现在 EX 段, 但我没有用寄存器存储 forward 信号而是产生之后直接用隧道传输到了 EX 段, 逻辑上有误。修改后通过。

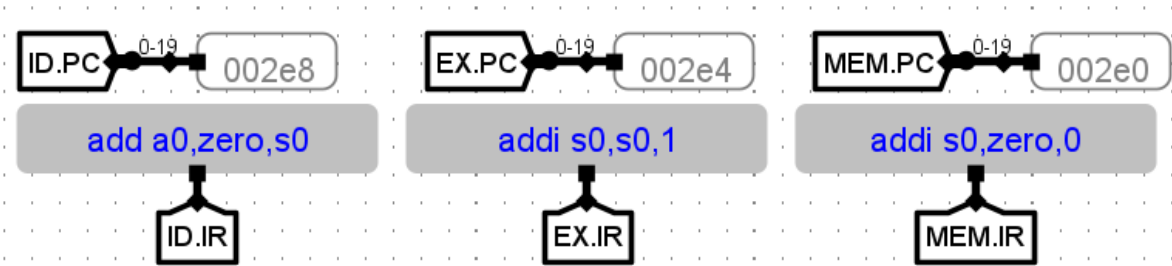


图 4.9 故障七. 1

信号	预期值 (Expected)	实际值 (Actual)	是否匹配	错误分析
IFPC	032f8	002f8	✗	严重错误: 取指阶段PC预期032f8, 实际为002f8 (高位缺失)
IDPC	32f4	02f4	✗	错误: 译码阶段PC预期32f4, 实际为02f4
EXPC	32f0	02f0	✗	错误: 执行阶段PC预期32f0, 实际为02f0
MPC	32ec	02ec	✗	错误: 访存阶段PC预期32ec, 实际为02ec
WBPC	32e8	02e8	✗	错误: 写回阶段PC预期32e8, 实际为02e8
PcErr	00000	-	-	PC错误标志未触发 (但实际PC值全错)
Werr	0100	-	-	总体错误 (检测到非PC错误)
RegW	1	1	✓	寄存器写使能正确
RDin	00000001	00000000	✗	关键错误: 预期写入数据0x01, 实际写入0x00
MemW	0	0	✓	内存写使能正确
MDin	00000000	00000000	✓	内存写入数据正确

图 4.10 故障七. 2

4.3.8 故障八

重定向流水线错误: MDin 错误, 运行到出错的节拍暂停, 发现是 sw 指令的 MDin 错误 (图 4.11 故障八.1), 而 MDin 的值来自经流水线向后传输的 R2 的值, 所以是 R2 的值错误。仔细检查电路, 发现重定向之后, 需要将 EX/MEM 的 R1R2 数据替换为重定向之后的 R1R2, 而我仍在用重定向之前的值, 将 R_EX 替换为重定向之后的值即可修复 bug (图 4.12 故障八.2)。

华中科技大学课程设计报告

CPU流水线信号对比表 (CLKs=0x344)

信号	预期值 (Expected)	实际值 (Actual)	是否匹配	错误分析
IFPC	031e0	001e0	✗	严重错误: 取指阶段PC应为001e0, 实际为031e0 (高位多出3)
IDPC	31dc	01dc	✗	错误: 译码阶段PC应为01dc, 实际为31dc
EXPC	31d8	01d8	✗	错误: 执行阶段PC应为01d8, 实际为31d8
MPC	31d4	01d4	✗	错误: 访存阶段PC应为01d4, 实际为31d4
WBPC	31d0	01d0	✗	错误: 写回阶段PC应为01d0, 实际为31d0
PcErr	00000	-	-	PC错误标志未触发 (但实际PC值全错)
Werr	0001	-	-	检测到非PC错误
RegW	1	1	✓	寄存器写使能正确
RDin	00000000	00000000	✓	寄存器写入数据正确
MemW	1	1	✓	内存写使能正确
MDin	ffffff	00000006	✗	关键错误: 内存写入数据应为ffffff, 实际为00000006

图 4.11 故障八. 1

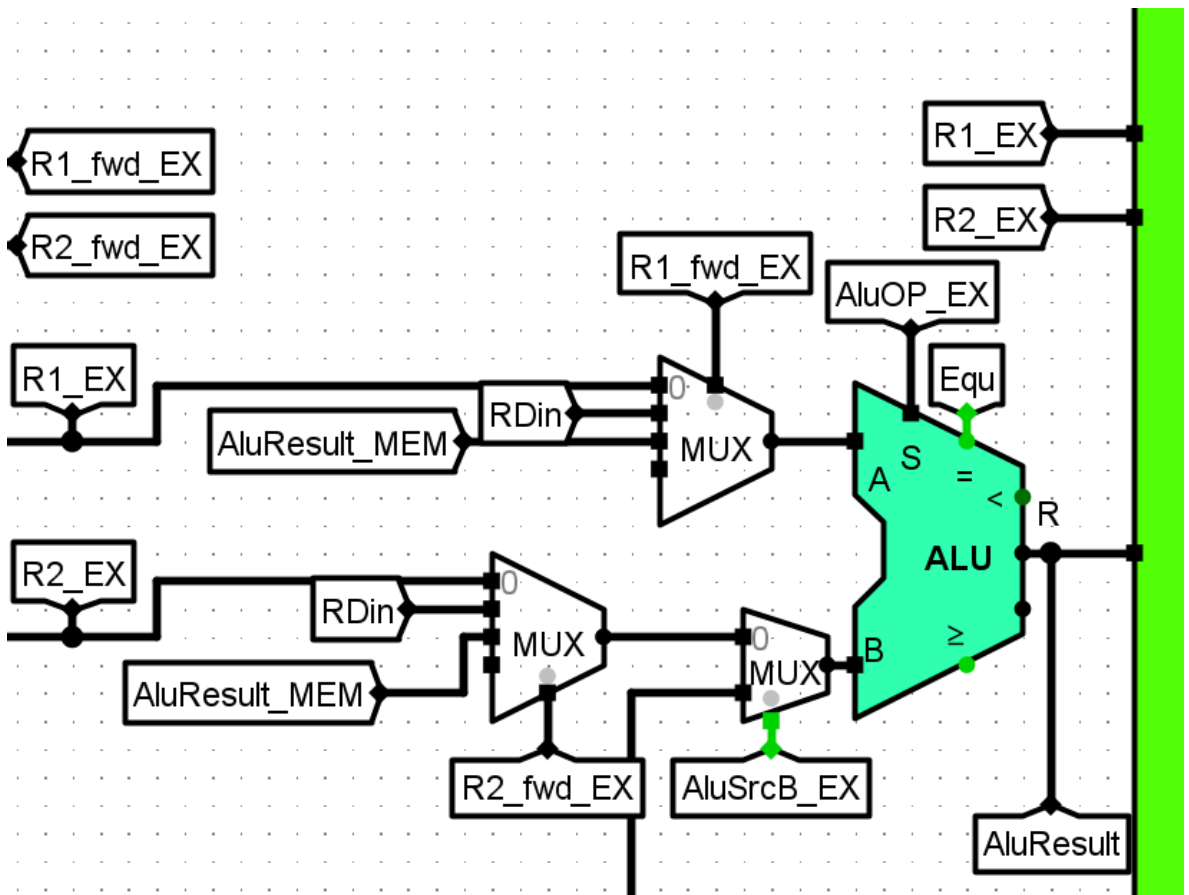
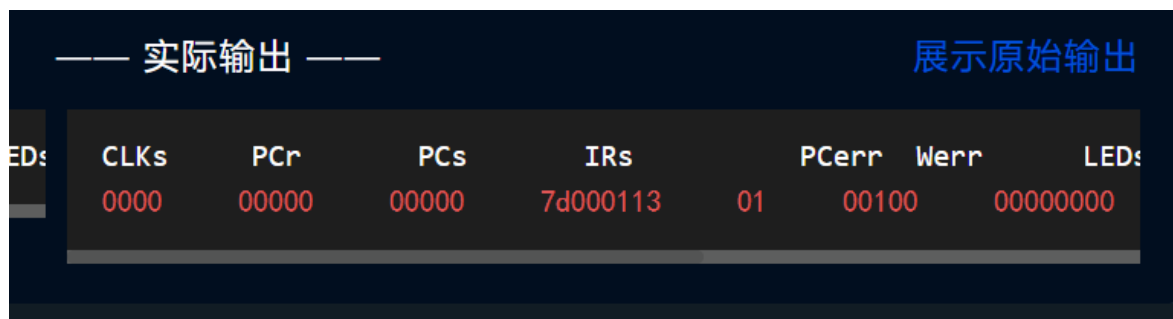


图 4.12 故障八. 2

4.3.9 故障九

单级中断第 0 拍就出错，指令寄存器忘记载入指令（图 4.13 故障九）。

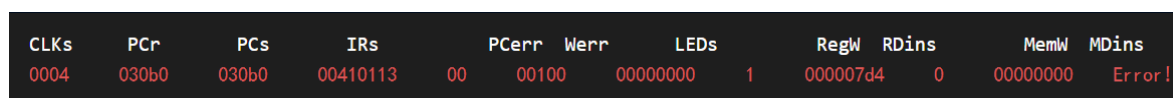


CLKs	PCr	PCs	IRs	PCerr	Werr	LEDs
0000	00000	00000	7d000113	01	00100	00000000

图 4.13 故障九

4.3.10 故障十

第 4 拍 RegW 出错（图 4.14 故障十），这是一条 sw 指令，但没有取出正确的寄存器的值，说明之前寄存器写入数据失败，仔细检查发现在流水线的关卡为解决数据冲突将寄存器设为下降沿触发，改为上升沿触发即可。



CLKs	PCr	PCs	IRs	PCerr	Werr	LEDs	RegW	RDins	MemW	MDins
0004	030b0	030b0	00410113	00	00100	00000000	1	000007d4	0	00000000 Error!

图 4.14 故障十

4.3.11 故障十一

动态分支预测流水线 CPU 停不下来，查看那一步 LED 灯的显示开始出错，最后定位到是某一个周期 BTB 中 next0 的值出错，进而发现是 BTB 的更新逻辑有误，应当是当 EX 阶段指令确实要跳转并且 BTB 没有命中才需要更新，而非 EX 阶段为跳转指令且 BTB 没有命中就更新。把 B_instruction 更换为 Branch 即可（图 4.15 故障十一）。

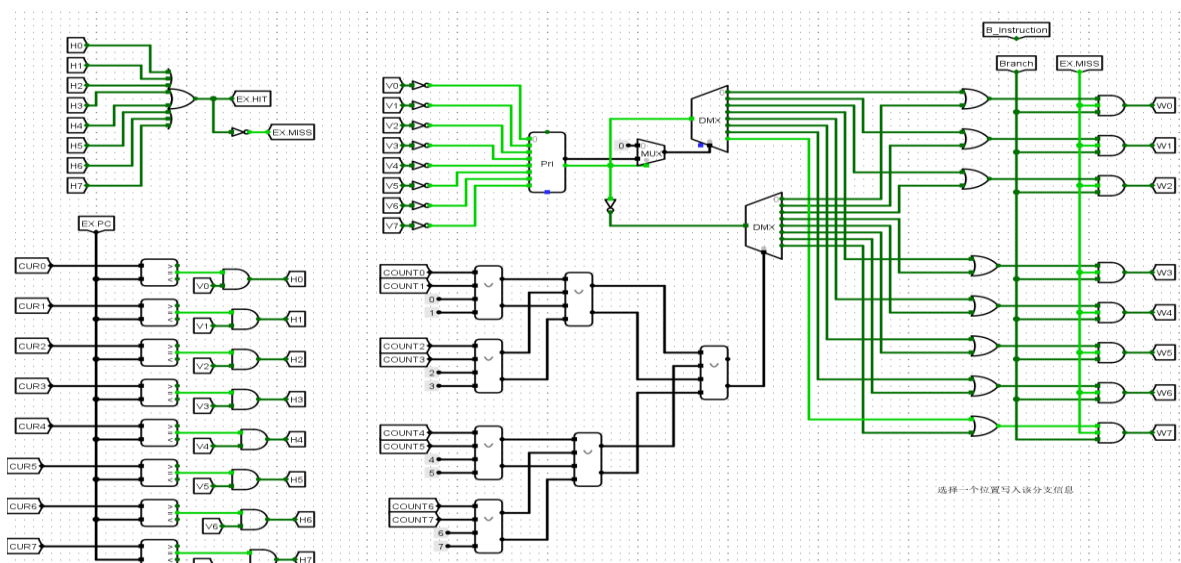


图 4.15 故障十一

4.4 实验进度

见表 4.3 课程设计进度表：

表 4.3 课程设计进度表

时间	进度
第一天	复习组成原理 CPU 相关理论知识，阅读课设任务书，阅读 MIPS 指令手册，并列出 CPU 各部件的数据通路表，并完成数据通路的基本构建。
第二天	完成单周期 CPU 的控制信号表，完成部分 Logism 单周期 CPU 故障报告。
第三天	完成 Logism 单周期 CPU 的故障报告。
第四天	完成了理想流水线。
第五天	完成气泡流水线的实验。
第六天	完成重定向流水线实验。
第七天	完成单级中断和多级中断单周期 CPU。
第八天	完成流水线中断。
第九天	开始动态分支预测流水线实验。
第十天	完成动态分支预测流水线实验。

5 团队任务

5.1 整体玩法概述

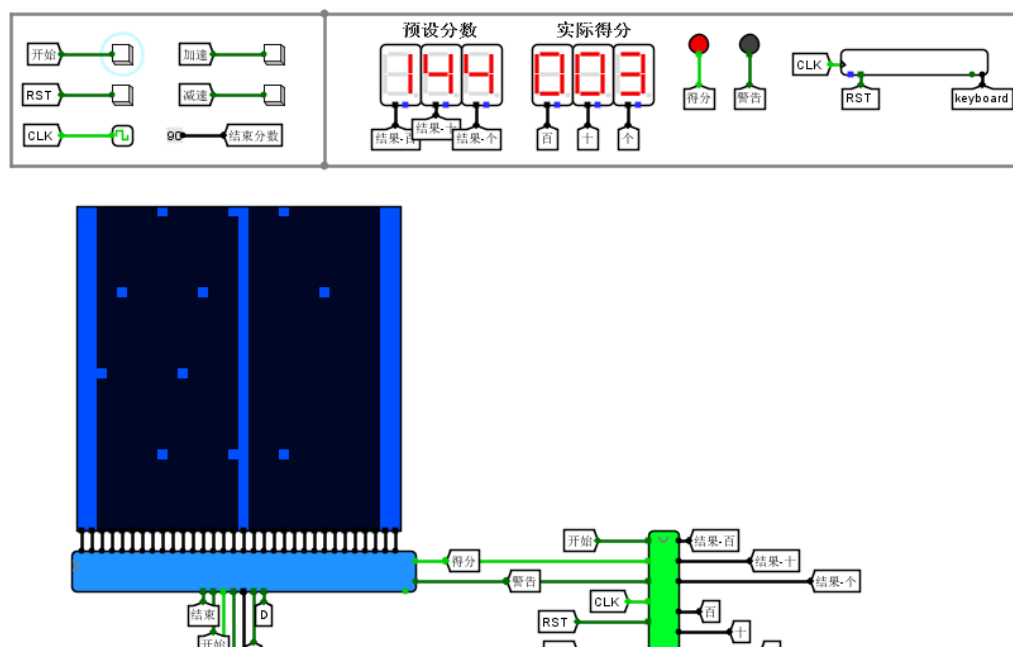


图 5.1 游戏展示 1



图 5.2 游戏展示 2

我们组设计的电路模拟的是一个太空战机在点阵屏幕上飞行并进行激光射击的互动游戏场景，如图 5.1 游戏展示 1、图 5.2 游戏展示 2。游戏开始后，玩家通过按键控制太空战机在点阵中左右移动位置，并在合适时机发射激光攻击前方出现的敌方目标。系统在时钟周期下实时检测激光与目标的位置关系，用于判断是否命中目标并进行计分。每次成功命中目标，分数累加，音乐响起。达到目标分数时，LED

亮、蜂鸣器长鸣、游戏结束。

5.2 核心组成逻辑

5.2.1 太空战机移动模块

1. 模块概述

太空战机移动控制模块是本系统人机交互的核心部分。该模块基于软硬件协同设计的思想，利用自主设计的单周期 CPU 及其三级中断机制，结合 MIPS 汇编软件逻辑，实现了战机的实时位移、边界穿梭（穿墙）、武器发射（激光）以及游戏启动前的倒计时引导功能。

2. 输入输出：

如图 5.3 移动模块：

输入：keyboard（键盘 A/D 控制左右移动）；速度（来自功能集合）；CLK（全局时钟）；RST（复位）。

输出：太空战机在 DotMatrix 点阵显示上的位置信号。太空战机的当前位置通过寄存器或计数器保存，当按下 A 键时位置减小（左移），按下 D 键时位置增大（右移）。

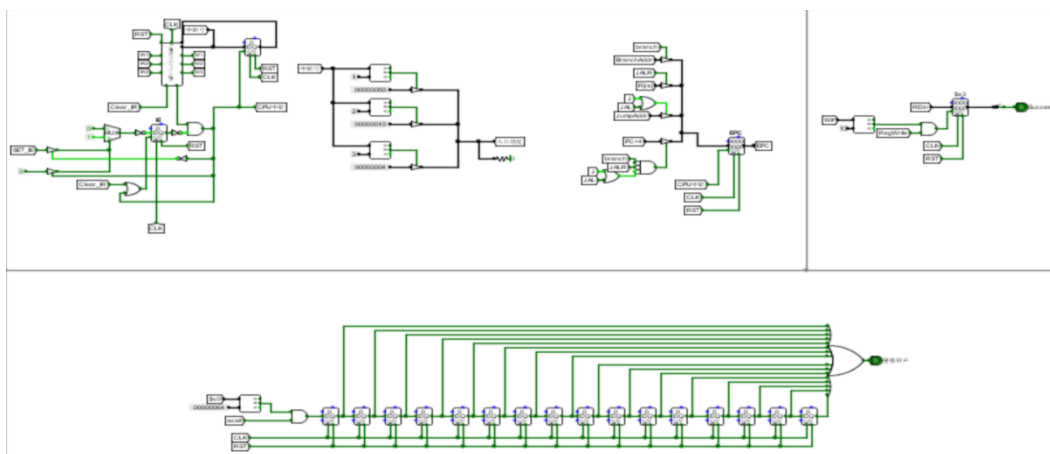


图 5.3 移动模块

3. 软硬件接口与显示原理

显示映射机制：系统采用内存映射 I/O (MMIO) 方式控制显示。显示屏被划分为若干列，每一个内存地址对应屏幕的一列像素（而非一行）。寄存器中的数值直接映射为该列像素的亮灭状态。

中断机制：RISCV CPU 支持三级硬件中断，本模块利用其中两级中断分别响应用户的“上移”和“下移”按键输入。中断信号一旦触发，CPU 立即挂起当前主循环，跳转至对应的中断服务程序 (ISR) 执行位移逻辑。

系统调用：通过自定义的 `ecall` 指令，软件向硬件发送屏幕刷新信号，确保显存数据的变更能实时呈现在显示设备上。

4. 软件逻辑详细设计

● 游戏初始化与倒计时逻辑

为提升用户体验，游戏在正式开始前引入了倒计时机制。

点阵绘制：根据列式显示的特性，程序预设了数字 3、2、1 的点阵数据。通过向屏幕中央的特定内存地址（如第 14-18 列）写入数据来绘制数字。

精准延时：考虑到 CPU 运行在 4.1kHz 的时钟频率下，程序通过空循环指令（`addi` 与 `bne` 组合）构建延时函数。经过调试，循环计数设定为 70 次，使得每个数字的显示时长约为 1 秒，实现了视觉上的流畅倒数。

场景构建：倒计时结束后，程序清除屏幕，绘制上下边界（地址 0-4 及 120-124），并将战机初始化在屏幕中央位置。

● 战机移动与“穿墙”机制

战机的移动逻辑完全由中断驱动，保证了操作的实时性。

位置管理：寄存器 `x9` 始终存储战机当前所在的内存地址（列坐标），寄存器 `x8` 存储战机的像素形态。

下移中断：当检测到下移信号时，程序首先判断 `x9` 是否达到下边界（地址 116）。若未达边界，则 `x9` 自增 4（移至下一列）；若已达边界，则触发“穿墙”逻辑，将 `x9` 强制重置为上边界起始位置（地址 8），实现循环移动效果。

上移中断：逻辑与下移类似，判断 `x9` 是否达到上边界（地址 8）。若未达边界，则 `x9` 自减 4；若已达边界，则将 `x9` 重置为下边界位置（地址 116）。

华中科技大学课程设计报告

状态同步：每次移动发生时，程序会强制清除当前的激光状态（若正在发射），并调用 syscall 34 刷新屏幕，防止残影产生。

● 武器系统（激光）的主循环逻辑

在没有中断发生时，CPU 执行主循环（nop_cycle），负责处理激光武器的自动发射与冷却。

状态机设计：利用寄存器 x11 作为计时器，x12 作为状态标志（0 为蓄力/冷却，1 为发射中）。

蓄力阶段：主循环每执行一次，计时器 x11 加 1。

发射阶段：当 x11 达到阈值（8 tick）且状态为未发射时，程序向当前战机位置 x9 写入全 1 数据（0xFFFFFFFF），点亮整列像素模拟激光发射，并将状态 x12 置 1。

持续与冷却：激光持续显示直到 x11 达到结束阈值（15 tick）。此时程序重绘战机原始形态（清除激光），并重置计时器与状态标志，进入下一轮循环。

5.2.2 目标生成与检测逻辑

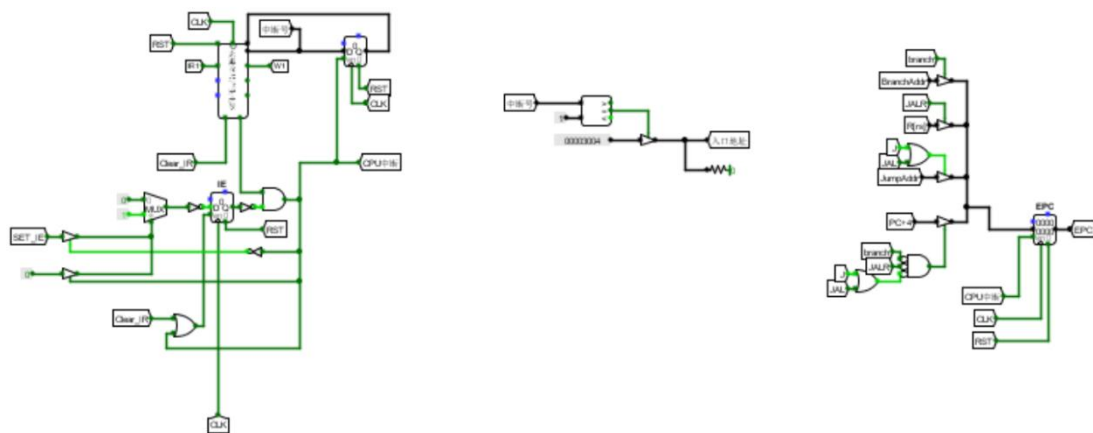


图 5.4 目标生成与检测

1. 模块概述

如图 5.4 目标生成与检测，敌机移动控制模块是游戏核心逻辑的重要组成部分，负责生成并控制敌机(原金币)在屏幕上的动态位移。该模块依托于自定义单周期 CPU 架构，通过软件编程实现敌机的分批生成、位置更新及循环刷新，营造出敌机群不断逼近的视觉效果。

2. 敌机生成与显示机制

系统采用列式内存映射显示，屏幕被划分为若干列，每列对应一个内存地址。敌机的形态由寄存器中的数值决定，通过向特定内存地址写入数据来在屏幕上绘制敌机。

分批生成策略：为了增加游戏难度与视觉层次感，敌机并非一次性全部出现，而是分三个阶段（s0, s1, s2）逐步生成。

初始阶段：首先在屏幕特定列（如 32, 60, 80）生成第一批敌机。

递增阶段：随着时间推移，第二批（如 16, 48, 128）和第三批（如 8, 40, 64）敌机依次加入战场，最终形成密集的敌机群。

3. 移动逻辑实现

敌机的移动通过位操作与循环控制实现，模拟了敌机从右向左（或特定方向）的飞行轨迹。

位移算法：利用逻辑左移指令（sll），将代表敌机位置的寄存器数值（x8, x9, x10）不断左移。由于显示映射机制，数值位的移动直接对应屏幕像素点的位移，从而实现敌机的平滑移动。

循环刷新：程序通过计数器（x12）控制移动步长（如每 8 步为一个周期）。在每个循环内，程序先将更新后的敌机位置写入显存，再调用系统调用（ecall 34）刷新屏幕，随后执行位移操作。

动态再生：当敌机群移动出屏幕或完成一个移动周期后，程序通过加法指令（addi）在寄存器低位补充新值，模拟敌机源源不断生成的动态效果，形成无限循环的敌机流（汇编代码中的 hhhh, iii, jjj 循环段）。

4. 游戏结束判定

虽然本模块主要负责移动，但也预留了游戏结束（finish）的逻辑接口。当满足特定条件（如被玩家击毁或碰撞）时，程序跳转至结束段，通过向屏幕中央写入特定的字符点阵数据（“END”字样），并刷新屏幕显示最终状态，标志着游戏流程的终结。

5.2.3 功能集合模块

功能集合模块承担以下几个关键任务（表 5.1 功能集合）：

表 5.1 功能集合

功能	实现方式	电路元件
得分计算	击中目标时计数器+1	计数器
分数比较	当前分数 vs 目标分数	比较器
游戏结束	当分数达到目标时输出结束=1	比较器+寄存器
速度调节	加速/减速按钮信号调节更新频率	复用器
警告提示	当接近目标分数时输出警告=1	二级比较器
音效触发	击中目标或结束时输出蜂鸣信号	Buzzer

5.2.4 玩家交互与反馈系统

在玩家交互电路中，玩家操作和系统反馈形成循环：

玩家操作层：按钮控制（开始/加速/减速/RST），键盘输入（A/D 移动）

游戏逻辑层：功能集合进行状态判断；战机移动模块更新画面；击中目标时触发计分与音效。

显示反馈层：DotMatrix 显示战机位置；数码管显示得分；LED 与 Buzzer 提供提示。点阵显示模块部分电路如下图 5.5 点阵显示：

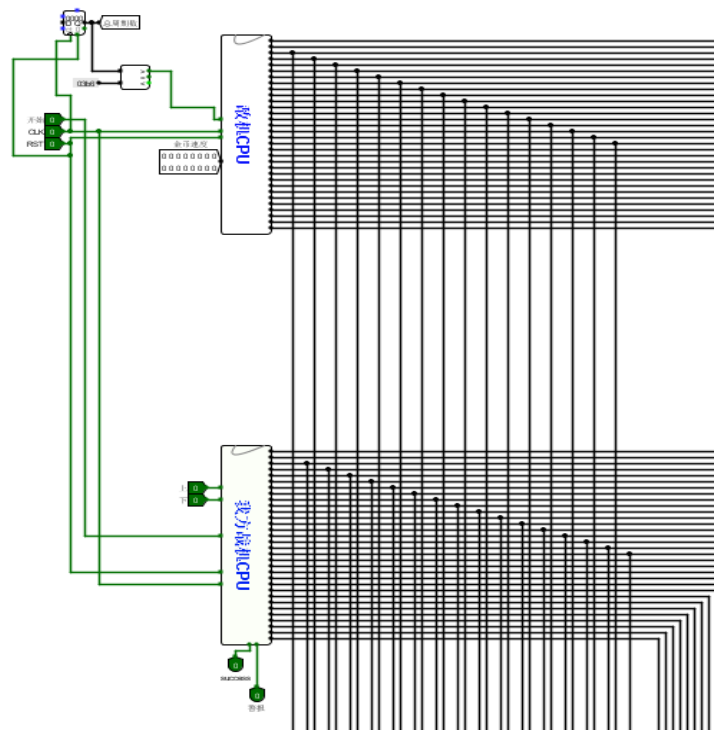


图 5.5 点阵显示

5.2.5 使用手柄控制游玩

传统上，Logisim 内的交互主要依赖键鼠接口。为了提升用户体验并验证多模态输入设备的兼容性，我们引入了第三方游戏控制器（手柄，型号为启明星 2）作为主要的操控手段。为了实现手柄控制，我们采用了三层抽象架构：

1. 物理层

设备接入： 游戏手柄通过 USB 接口接入操作系统，被识别为标准 HID (Human Interface Device) 设备。

原生数据流： 手柄驱动程序将物理按键按下、摇杆偏移等事件转换为操作系统可理解的原始输入事件。

2. 抽象层

JoyToKey 作为中间件： JoyToKey 在此架构中扮演了输入抽象中间件的角色。

关键工作： 我们的核心工作是精密配置其内部的输入配置文件，以实现逻辑按键组到虚拟键鼠事件的转换。例如，将手柄的方向键转换为离散的数字信号，最终映

射为虚拟键盘上的 A/D 按键事件。具体映射如表 5.2 手柄映射。

表 5.2 手柄映射

Logisim 逻辑功能	手柄物理按键/轴	抽象层虚拟事件
向左移动	POV:←	虚拟键盘 A
向右移动	POV:→	虚拟键盘 D
移动光标	stick←← ↑ →	虚拟鼠标
确认	A	模拟点击
开启时钟	LT+B	虚拟键盘 ctrl+k

难点体现： 精确校准手柄死区（Deadzone）和响应曲线，确保摇杆微小偏移不会触发意外操作，同时保证最大偏移时的响应速度，这涉及对中间件配置参数的量化调整。

调整界面如图 5.6 映射调整：

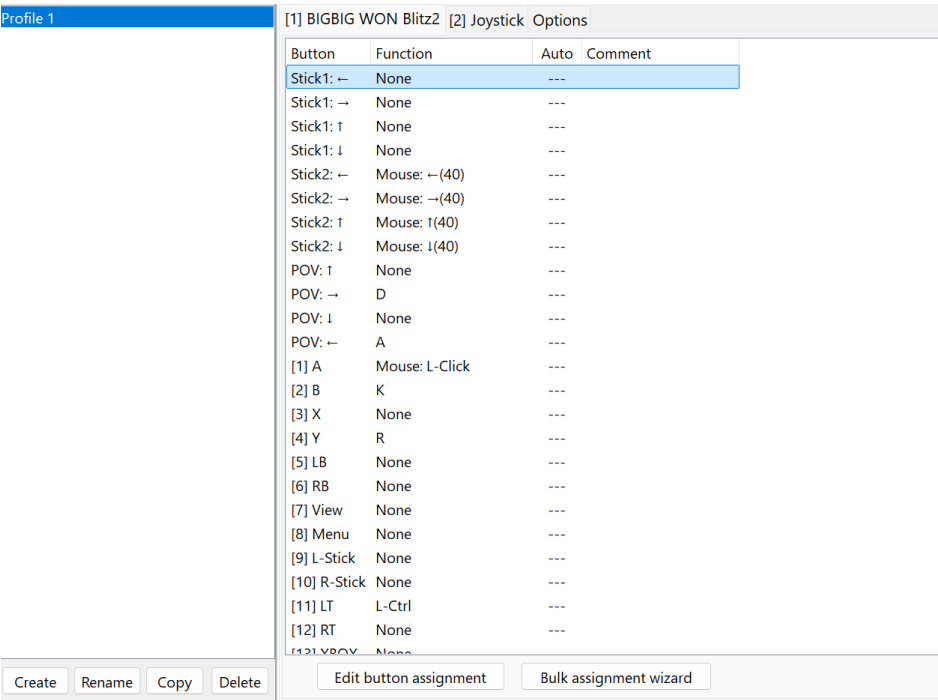


图 5.6 映射调整

3. 目标层

Logisim 接收端： Logisim 仿真环境将接收到的虚拟键盘事件（如 'A','D'）转换为其内部元件的数字逻辑输入信号。

5.2.6 游戏流程串联

游戏上电后，系统首先进入初始化阶段。复位信号（RST）对所有模块生效，将计数器清零、分数寄存器置为 0，确保系统处于初始稳定状态；

当玩家按下开始按钮后，全局时钟（CLK）被激活，游戏正式开始。开始信号传入功能集合模块，同时使太空战机移动模块进入工作状态；

在游戏运行过程中，玩家通过键盘输入控制太空战机在点阵屏幕中的左右移动。键盘信号用于更新战机位置寄存器，新的位置信息实时输出至 DotMatrix 点阵显示模块，用于显示太空战机的当前飞行位置；

当太空战机发射激光并成功命中敌方目标时，系统通过位置重合检测逻辑产生命中信号。该信号传入功能集合模块，用于驱动分数累加，从而完成一次有效攻击的计分过程。分数变化后，当前得分会经过计数与进制转换模块处理，并以十六进制形式输出到数码管上，实现分数的实时显示；

当当前分数接近或达到预设目标分数时，功能集合模块内部的比较器被触发，系统进入警告阶段。此时 LED 开始闪烁，同时蜂鸣器发出提示音，用于提醒玩家即将完成游戏目标；

当分数最终达到目标值后，系统输出游戏结束信号，全局时钟停止工作，太空战机与激光运动终止，同时播放结束音效，游戏流程结束。

5.3 组员分工情况

组员具体分工情况如表 5.3 分工：

表 5.3 分工

分工	组员
系统总体架构设计	李邵永
电路搭建	王希宇 王一然
汇编代码编写	王一然
系统串联	孙啸啸
报告撰写	李邵永 王一然

6 设计总结与心得

6.1 课设总结

通过为期两周的硬件综合训练，我们成功地设计并实现了一个支持 RISC-V 指令集的 5 段流水线 CPU。以下是本次工作的全面总结：

- 1) 完成了单周期 CPU（支持 24 条基础指令和 4 条差异化指令）、理想流水线 CPU、气泡流水线 CPU、重定向流水线 CPU、动态分支预测流水线 CPU、单级中断和多级嵌套中断单周期 CPU，单级中断流水线 CPU。
- 2) 实现了以下功能：
 - 指令集支持：完整支持规定的 24 条基本 RISC-V 指令，并成功扩展了 CCAB 系列中的多条指令。
 - 流水线与冒险处理：实现了 5 段流水线，能够正确处理数据冒险（通过转发与气泡）、控制冒险（通过分支预测与冲刷），并具备相应的冲突次数统计功能。
 - 中断机制：实现了多级嵌套中断，支持 3 个外部中断源，并能依据固定优先级进行响应与嵌套。
 - 动态分支预测：实现了基于 BTB 和两位饱和计数器的动态分支预测器，并能自动统计预测成功与失败次数。
 - 正确性验证：所设计的 CPU 能够在 Logisim 平台上正确运行教师提供的标准测试程序及自编测试程序，并通过了头歌平台的自动化评测。

6.2 课设心得

本次硬件综合训练课程设计，是我大学阶段一次极具挑战性也最具成就感的实践经历。当看到自己亲手设计的 CPU 最终能够正确地执行复杂的测试程序时，所有的艰辛与付出都化为了喜悦。回首整个设计过程，我收获的不仅仅是一个能够运行的流水线 CPU，更是一次认知与能力的全面升华。

第一，对计算机系统的认识从未如此深刻。在以往的课程中，CPU、中断、流水线这些概念于我而言，更多是教科书上抽象的框图与文字描述。而本次设计，让我得

华中科技大学课程设计报告

以“亲手触摸”这些概念的实体。这一切让我深刻地认识到，一台现代计算机的奇迹，正是由无数个如此精妙而基础的电路模块协同构建而成的。

第二，我的工程实践与解决问题能力得到了实质性的锻炼。这次课设是一次完整的“造轮子”过程，极大地培养了我的系统设计能力和严谨的工程思维。从顶层的方案规划，到底层的每一个逻辑门的实现，都需要周密的思考。更重要的是，我学会了如何与“故障”共处。调试占据了项目的大部分时间，从最初面对错误时的茫然无措，到后来能熟练运用 Logisim 的单步调试、利用头歌平台的错误信息精准定位问题，我的故障分析与定位能力得到了质的飞跃。我明白了，一个优秀的工程师不仅在于能设计出正确的电路，更在于当电路出现问题时，拥有抽丝剥茧、直指核心的调试能力。

第三，我由衷地见识并感叹计算机体系结构之美。这种美，是一种严谨、高效与和谐的统一。它体现在流水线如同精密的工业流水线般，让指令处理并行不悖；体现在数据转发机制如同巧妙的捷径，化解了数据的冒险；也体现在动态分支预测仅仅依靠硬件就能完成对分支的预测。整个 CPU 结构层层递进，环环相扣，各个部件在时钟信号的同步下，演奏出一曲协调有序的交响乐。这种由简单逻辑门构建出的复杂智能，这种在严格规则下涌现出的高效与灵活，也让我对计算机科学的开创者们充满了敬意。

另外我建议在课程指导中加入动态分支预测的相关章节，因为这个实验无疑是较难的，但指导书里并没有太多提示。

总而言之，这门课不仅赋予我将理论付诸实践的机会，更在我心中种下了对底层硬件系统持续探索的热情。它是我理解计算机如何“计算”的里程碑，这段宝贵的经历将激励我在未来的学习和工作中，不断深入，追求卓越。

参考文献

- [1] DAVID A. PATTERSON(美). 计算机组成与设计硬件/软件接口(原书第 5 版). 北京: 机械工业出版社.
- [2] David Money Harris(美). 数字设计和计算机体系结构(第二版). 机械工业出版社
- [3] 谭志虎, 秦磊华, 吴非, 等. 计算机组成原理(微课版 第 2 版). 北京: 人民邮电出版社, 2025 年.
- [4] 谭志虎, 周健, 周游. 计算机组成原理实验指导(基于 RISC-V 在线实训). 北京: 人民邮电出版社, 2024 年.
- [5] 曹强, 施展. 计算机系统结构(微课版). 北京: 人民邮电出版社, 2024 年.

• 指导教师评定意见 •

一、原创性声明

本人郑重声明本报告内容，是由作者本人独立完成的。有关观点、方法、数据和文献等的引用已在文中指出。除文中已注明引用的内容外，本报告不包含任何其他个人或集体已经公开发表的作品成果，不存在剽窃、抄袭行为。

特此声明！

作者签字：王希宇